

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



PROJECT REPORT

Đề tài:

**THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG THU THẬP,
TRUY VẾT VÀ PHÂN TÍCH SỰ KIỆN MẠNG (LOG)
TẬP TRUNG DỰA TRÊN KIẾN TRÚC SERVERLESS CLOUD**

Sinh viên thực hiện: Nguyễn Ngọc Linh – 20235599
Học phần: Project II
Mã lớp: 761985
Giảng viên hướng dẫn: Mai Xuân Ngọc

Hà Nội, Năm 2026

ABSTRACT

(TÓM TẮT)

Trong các hệ thống mạng doanh nghiệp hiện nay, mặc dù đã có các giải pháp phát hiện xâm nhập (như IDPS), việc quản lý nhật ký sự kiện (log) phân tán trên từng máy chủ cục bộ vẫn bộc lộ nhiều hạn chế. Khi xảy ra sự cố bảo mật, quản trị viên mất rất nhiều thời gian để truy cập thủ công vào từng thiết bị nhằm truy vết nguyên nhân gốc rễ (root cause analysis). Hơn nữa, dữ liệu log lưu tại máy có nguy cơ cao bị tin tặc xóa bỏ để che giấu hành vi sau khi chiếm quyền.

Đề án này tập trung phân tích, thiết kế và xây dựng một “*Hệ thống Quản lý Log Tập trung*” áp dụng hoàn toàn kiến trúc Serverless trên nền tảng Amazon Web Services (AWS). Giải pháp này giúp doanh nghiệp loại bỏ sự phụ thuộc vào các máy chủ lưu trữ (như Syslog server) vật lý vốn đắt đỏ và dễ bị thất nút cổ chai. Hệ thống sử dụng Amazon API Gateway và AWS Lambda để kiểm duyệt luồng dữ liệu (qua IP và Token kép), lưu trữ an toàn trên NoSQL Amazon DynamoDB. Giao diện quản trị tính được phân phối qua mạng viền Amazon CloudFront (HTTPS). Bằng cách thiết lập luồng giao tiếp một chiều đẩy log theo thời gian thực lên đám mây, hệ thống bảo vệ dữ liệu hoàn toàn khỏi thiết bị gốc, đảm bảo tính toàn vẹn ngay cả khi máy chủ bị chiếm quyền. Đề án mong muốn mang lại cho tổ chức một giải pháp chịu tải đột biến tốt, tối ưu chi phí (Pay-as-you-go) và cung cấp công cụ giám sát, điều tra toàn diện.

MỤC LỤC

ABSTRACT (TÓM TẮT)	1
1 INTRODUCTION (GIỚI THIỆU TỔNG QUAN)	4
1.1 Bối cảnh và Vấn đề	4
1.2 Giải pháp Kiến trúc Serverless (Điện toán vô máy chủ)	4
1.3 Mục tiêu Đồ án	6
2 PLANNING (LẬP KẾ HOẠCH DỰ ÁN)	9
2.1 Yêu cầu Hệ thống (System Request)	9
2.2 Phân tích Tính Khả thi (Feasibility Analysis)	9
2.2.1 Khả thi về Kỹ thuật:	9
2.2.2 Khả thi về Kinh tế (Economic Feasibility):	10
2.2.3 Khả thi về Tổ chức:	11
2.3 Lựa chọn Phương pháp Phát triển (Development Methodology)	11
3 SYSTEMS ANALYSIS (PHÂN TÍCH HỆ THỐNG VÀ YÊU CẦU)	12
3.1 Phân tích Yêu cầu Chức năng (Functional Requirements)	12
3.1.1 Phân hệ Quản trị & Xác thực (Admin Authentication)	12
3.1.2 Phân hệ Quản lý Thiết bị (Device Management)	12
3.1.3 Phân hệ Tiếp nhận & Định tuyến Dữ liệu (Log Ingestion & Routing)	12
3.1.4 Phân hệ Giao diện Giám sát & Điều tra (Web Dashboard)	12
3.2 Phân tích Yêu cầu Phi Chức năng (Non-Functional Requirements)	13
3.3 Biểu đồ Ca sử dụng (Use Case Diagram)	13
3.4 Đặc tả Ca sử dụng (Use Case Specification)	14
3.4.1 UC-01: Đăng nhập (Login)	14
3.4.2 UC-02: Giám sát & Xử lý Cảnh báo Real-time	15
3.4.3 UC-03: Tra cứu Lịch sử (Historical Inquiry)	16
3.4.4 UC-04: Thêm Thiết bị mới (Add Device)	17
3.4.5 UC-05: Sửa Thông tin Thiết bị (Edit Device Info)	18
3.4.6 UC-06: Xóa Thiết bị (Delete Device)	19
3.4.7 UC-07: Thu thập và Phân loại Log (Log Collection & Classification)	20

4	SYSTEMS DESIGN (THIẾT KẾ HỆ THỐNG)	22
4.1	Kiến trúc Hệ thống Tổng thể (System Architecture)	22
4.2	Biểu đồ Tuần tự (Sequence Diagram)	24
4.3	Các Cơ chế Bảo vệ của Hệ thống (Security Mechanisms)	29
4.3.1	Bảo vệ lớp Frontend và Mạng truyền tải (S3 & CloudFront)	29
4.3.2	Bảo vệ lớp API và Quản lý Lưu lượng (API Gateway)	30
4.3.3	Bảo vệ lớp Tính toán và Dữ liệu (Lambda & DynamoDB)	31
4.4	Phương án Tích hợp Message Queue bằng Amazon SQS	32
4.4.1	Giới thiệu và Mục đích sử dụng Amazon SQS	32
4.4.2	Đánh giá tính phù hợp và Bài toán chi phí	33
5	TRIỂN KHAI VÀ CẤU HÌNH HỆ THỐNG (SYSTEM IMPLEMENTATION)	35
5.1	Cấu hình Agent tại máy trạm (Edge Endpoint Configuration)	35
5.1.1	Lọc và Trích xuất dữ liệu tại nguồn (Edge-Filtering)	35
5.1.2	Đóng gói dữ liệu (Batching)	36
5.1.3	Cơ chế Xác nhận (ACK) và Quản lý tọa độ đọc	36
5.1.4	Xử lý Lỗi (Error Handling) và Thoát an toàn	36
5.2	Triển khai Hạ tầng Lưu trữ (Storage Tier)	37
5.2.1	Cấu hình Cơ sở dữ liệu Amazon DynamoDB	37
5.2.2	Cấu hình Hạ tầng Khởi tạo Amazon S3 Bucket	39
5.3	Triển khai Tầng Xử lý (Compute Tier)	40
5.3.1	Cấu hình Hạ tầng Quản lý Định danh Amazon Cognito	41
5.3.2	Phát triển và Triển khai các Hàm AWS Lambda	41
5.3.3	Thiết lập Phân quyền qua AWS IAM Roles	42
5.4	Triển khai Tầng Mạng biên (Edge Tier)	46
5.4.1	Cấu hình API Gateway và Định tuyến	46
5.4.2	Thiết lập Bộ lọc IP (IP Whitelisting) qua Resource Policy	47
5.4.3	Tích hợp Trình xác thực Amazon Cognito Authorizer	48
5.4.4	Thiết lập Amazon CloudFront và Bảo mật SSL/HTTPS	48
6	TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN (CONCLUSION & FUTURE WORK)	50
6.1	Kết luận	50
6.2	Hướng phát triển	51

CHƯƠNG 1.

INTRODUCTION (GIỚI THIỆU TỔNG QUAN)

1.1 Bối cảnh và Vấn đề

Mạng nội bộ của doanh nghiệp luôn là đích đến của các cuộc tấn công mạng như dò quét cổng (Nmap Scan), tấn công bạo lực (SSH Brute-force) hay lây nhiễm mã độc,... Khi có sự cố, hệ thống nhật ký (Log) là nguồn bằng chứng duy nhất để điều tra (Forensics). Tuy nhiên, nếu log chỉ được lưu tại ổ cứng cục bộ, tin tặc sau khi xâm nhập thành công có thể dễ dàng xóa sạch dấu vết. Mặt khác, nếu xây dựng các máy chủ vật lý tập trung, doanh nghiệp lại vấp phải rủi ro sập hệ thống (thắt nút cổ chai) khi lượng log đổ về tăng đột biến trong các cuộc tấn công DDoS.

1.2 Giải pháp Kiến trúc Serverless (Điện toán vô máy chủ)

Khác với các mô hình hạ tầng truyền thống dựa trên định dạng IaaS (Infrastructure as a Service) – nơi doanh nghiệp buộc phải duy trì liên tục các máy chủ ảo 24/7 chỉ để trực chờ tiếp nhận dữ liệu nhật ký sự kiện từ hệ thống – kiến trúc Serverless mang lại một sự chuyển dịch căn bản mang tính đột phá về cả phương thức vận hành, khả năng chịu tải lẫn mô hình tối ưu chi phí. Sự vượt trội này được thể hiện rõ nét qua bốn khía cạnh cốt lõi sau:

- **Loại bỏ hoàn toàn gánh nặng vận hành hạ tầng (No Server Management):** Trong mô hình máy chủ truyền thống, người quản trị phải chịu trách nhiệm toàn bộ vòng đời của hệ thống bao gồm: cài đặt hệ điều hành, cấu hình môi trường chạy phần mềm, theo dõi dung lượng ổ cứng, và thực hiện vá các lỗ hổng bảo mật định kỳ (patching). Đối với kiến trúc Serverless, toàn bộ các lớp phần cứng ảo hóa, hệ điều hành (OS) và môi trường thực thi đều do nhà cung cấp đám mây (AWS) quản lý toàn diện. Quản trị viên chỉ cần tập trung hoàn toàn vào việc tối ưu hóa logic xử lý dữ liệu log của mã nguồn mà không phải lo lắng về các sự cố sập phân vùng đĩa cứng vật lý hay xung đột thư viện hệ thống.
- **Khả năng tự động co giãn tức thời và toàn diện (Full-Stack Auto-scaling):** Điểm yếu lớn nhất của các máy chủ log truyền thống (như Syslog Server) là dễ rơi vào trạng thái nghẽn cổ chai hoặc sập đổ hoàn toàn khi xảy ra sự kiện tấn công mạng (như bão log sinh ra từ các cuộc tấn công từ chối dịch vụ DDoS). Kiến trúc Serverless xử lý triệt để vấn đề này nhờ khả năng

năng tự co giãn đồng bộ ở cả ba tầng:

- *Tầng xử lý mạng biên (Amazon API Gateway)*: API Gateway tự động tiếp nhận và điều phối hàng vạn kết nối đồng thời mà không yêu cầu thiết lập bộ cân bằng tải (Load Balancer) chuyên dụng, tích hợp sẵn cơ chế điều tiết lưu lượng (Throttling) sử dụng thuật toán Token Bucket cho phép thiết lập giới hạn tốc độ truy cập (Requests Per Second - RPS) và dung lượng bùng nổ (Burst limit) theo từng định danh máy trạm (thông qua Usage Plans & API Keys) hoặc trên toàn bộ luồng API. Khi một tác nhân gửi lượng request vượt quá hạn mức cho phép, API Gateway sẽ lập tức chặn đứng khối giao thông đó và trả về mã lỗi HTTP 429 (Too Many Requests) ngay tại lớp biên, bảo vệ tầng xử lý phía sau và tối ưu hóa chi phí vận hành.
- *Tầng xử lý logic (AWS Lambda)*: AWS Lambda hoạt động dựa trên công nghệ ảo hóa vi mô (MicroVM) mã nguồn mở **Firecracker**. Mỗi khi có dữ liệu nhật ký gửi lên, Lambda khởi tạo một môi trường thực thi (Execution Environment) hoàn toàn biệt lập về CPU và RAM chỉ trong vài chục mili-giây. Khi xảy ra hiện tượng bão log (Log Storms), cơ chế **Concurrent Executions** (thực thi đồng thời) lập tức được kích hoạt. Lambda tự động nhân bản ngang (scale out) thành hàng nghìn MicroVM chạy song song để xử lý từng request độc lập, hấp thụ hoàn toàn lưu lượng tải khổng lồ. Nhờ cơ chế **Scale-to-Zero**, ngay khi kết thúc xử lý, toàn bộ các MicroVM này sẽ tự động bị tiêu hủy, đưa lượng tài nguyên tiêu thụ và chi phí vận hành về đúng mức 0, loại bỏ lãng phí của các máy chủ chạy không tải (Idle).
- *Tầng cơ sở dữ liệu (Amazon DynamoDB)*: Cấu hình ở chế độ **On-Demand Capacity Mode** loại bỏ hoàn toàn việc dự trữ tài nguyên bằng cơ chế tự động mở rộng thông lượng tức thì lên tới **gấp đôi đỉnh tải (peak) cao nhất trước đó**, với thông lượng khởi tạo mặc định là **4.000 WCU** và **12.000 RCU** mỗi giây, được **chia đều** cho các phân mảnh vật lý (Partitions). Khi xảy ra bão log khiến một dải lưu lượng làm quá tải một phân mảnh (Hot Partition), DynamoDB tự động kích hoạt **Adaptive Capacity** (Năng lực thích ứng) để luân chuyển thông lượng dư thừa từ các phân mảnh rảnh rỗi sang, giúp phân mảnh bị có thể hoạt động lên tới mức tối đa là **1.000 WCU** và **3.000 RCU**. Nếu lưu lượng tiếp tục bùng nổ vượt qua mức giới hạn vật lý này, hệ thống sẽ ngay lập tức thực thi **Partition Splitting** (Chia tách phân mảnh) để chia phân mảnh đó sang máy chủ mới, tự động nhân đôi thông lượng đỉnh WCU, RCU nhằm đảm bảo quá trình ghi nhật ký diễn ra liên tục.
- *Tầng lưu trữ tĩnh (Amazon S3)*: Kiến trúc S3 quản lý thông lượng dựa trên **Tiền tố đối tượng (Object Prefix)**. Mỗi một tiền tố (ví dụ: /static/js/) được phân bổ mặc định thông lượng **3.500 yêu cầu ghi (PUT/POST)** và **5.500 yêu cầu đọc (GET/HEAD)** mỗi giây. Khi lượng truy cập Dashboard tăng vọt, S3 liên tục giám sát các mẫu truy cập (Access Patterns). Nếu một tiền tố có dấu hiệu quá nhiệt (vượt ngưỡng 5.500 GET/s),

S3 tự động kích hoạt cơ chế **Automatic Partitioning**. Quá trình này diễn ra ngầm bằng cách chia tách bảng chỉ mục (Index Splitting) của tiền tố đó theo trật tự từ vựng học (Lexicographical order) và điều hướng các dải Object Key phân tán sang các cụm máy chủ (Nodes) độc lập khác nhau. Nhờ cơ chế chế nhánh tải trọng tự động này, thông lượng giới hạn được nhân lên nhiều lần, đảm bảo mã nguồn ReactJS luôn tải xuống ổn định mà không bị nghẽn cổ chai hạ tầng.

- **Tối ưu hóa chi phí dựa trên hiệu năng thực tế (Pay-as-you-go Financial Model):** Đối với hệ thống IaaS truyền thống, doanh nghiệp phải trả một khoản chi phí cố định theo tháng cho CPU, RAM và dung lượng ổ cứng của máy chủ ảo ngay cả khi hệ thống nhàn rỗi không có log gửi về (lãng phí tài nguyên nhàn rỗi). Ngược lại, kiến trúc Serverless áp dụng mô hình tính phí theo mức độ sử dụng thực tế (Pay-as-you-go). Hệ thống chỉ tính toán chi phí khi có mã nguồn thực thi: AWS Lambda tính phí chính xác dựa trên số lượng yêu cầu kích hoạt và số mili-giây xử lý thực tế; Amazon DynamoDB ở chế độ On-Demand chỉ tính tiền dựa trên số lượng dòng dữ liệu thực tế được ghi/đọc xuống ổ đĩa vật lý; Amazon S3 tính phí trên số GB lưu trữ thực tế theo tháng. Mô hình này giúp doanh nghiệp triệt tiêu hoàn toàn chi phí duy trì phần cứng nhàn rỗi vào ban đêm hoặc trong những khoảng thời gian hệ thống mạng an toàn.
- **Thu hẹp tối đa bề mặt tấn công hệ thống (Minimized Attack Surface):** Một máy chủ truyền thống muốn tiếp nhận log hoặc cho phép quản trị viên cấu hình thì bắt buộc phải mở công khai các cổng dịch vụ mạng (như Port 22 cho SSH, Port 80/443 cho Web, Port 514 cho Syslog). Đây là những mục tiêu hàng đầu để tin tặc tiến hành rà quét lỗ hổng dịch vụ, tấn công bạo lực (Brute-force) hoặc khai thác lỗi tràn bộ đệm hệ điều hành nhằm chiếm quyền kiểm soát máy chủ cục bộ. Trong kiến trúc Serverless, toàn bộ bề mặt tấn công này bị xóa bỏ. Hệ thống không sở hữu địa chỉ IP công khai gắn liền với từng dịch vụ, không có hệ điều hành phơi bày ra Internet để tin tặc quét lỗi, và không có các cổng quản trị mở sẵn. Điểm tiếp xúc duy nhất với Internet là các API Endpoint được mã hóa và kiểm duyệt chặt chẽ bởi các chính sách bảo mật biên của AWS, ngăn chặn hoàn toàn nguy cơ tin tặc tấn công sâu vào hạ tầng lõi hoặc cài cắm webshell nhằm duy trì sự hiện diện lâu dài.

1.3 Mục tiêu Đề án

Để giải quyết triệt để các bài toán thực tế về quản lý dữ liệu an ninh mạng tập trung, đề án đặt ra bốn mục tiêu cốt lõi với cấu trúc phân tầng hạ tầng kỹ thuật rõ ràng:

- **Thứ nhất, thiết lập kênh truyền tải an toàn và bảo mật ranh giới mạng biên:** Xây dựng giải pháp tiếp nhận nhật ký sự kiện từ các thiết bị ngoại vi thông qua kiến trúc mạng biên bảo mật. Sử dụng Amazon API Gateway kết hợp với Resource Policy để triển khai cơ chế lọc và cô lập giao thông mạng (IP Whitelisting), chỉ cho phép các địa chỉ IP thuộc dải mạng

nội bộ hợp lệ của doanh nghiệp kết nối. Đồng thời, xây dựng cơ chế xác thực đa lớp tại tầng tiếp nhận bằng cách kiểm tra cặp khóa định danh thiết bị nghiêm ngặt (Machine-Token và Machine-Secret) đính kèm trong HTTP Header của gói tin gửi từ Agent, chặn các nguy cơ giả mạo dữ liệu nhét ký sự kiện (Log Spoofing) từ các thiết bị lạ.

- **Thứ hai, phát triển kiến trúc logic xử lý trung tâm phân tán (Backend Logic Tier):**

Phát triển bộ mã nguồn xử lý phân tán dựa trên nền tảng AWS Lambda, chia tách hệ thống thành ba cụm điều khiển độc lập (Microservices Controller) tuân thủ mô hình kiến trúc hướng sự kiện nhằm đảm bảo khả năng cô lập lỗi:

- LogIngestionController: Đảm nhận nhiệm vụ bóc tách mảng JSON tuần tự với độ phức tạp $O(N)$, sử dụng các biểu thức chính quy (Regex) để phân loại dòng dữ liệu thô từ Suricata, UFW, Syslog thành các trường dữ liệu có cấu trúc rồi đẩy xuống cơ sở dữ liệu. Module này hoạt động theo nguyên tắc ghi một chiều (Non-destructive write-only), không cấp quyền chỉnh sửa hay xóa để bảo vệ bằng chứng số.
- QueryController: Chứa các tiến trình RealtimeLogsFetcher (truy vấn nhanh 50 bản ghi mới nhất hiển thị tự động lên UI) và AdvancedSearchAPI (xử lý các bộ lọc động phục vụ công tác điều tra).
- ManagerController: Quản lý các tác vụ quản trị chuyên sâu như cập nhật trạng thái cảnh báo và quản lý danh sách thiết bị với cơ chế độ trễ an toàn 15 giây chống thao tác sai.

Toàn bộ cụm điều khiển này được bảo vệ bằng phân hệ quản lý danh tính ****Amazon Cognito****, bắt buộc kiểm tra mã hóa chuỗi JSON Web Token (JWT) thông qua bộ kiểm duyệt tự động Cognito Authorizer tại API Gateway trước khi cho phép mã nguồn khởi chạy.

- **Thứ ba, thiết kế mô hình cơ sở dữ liệu NoSQL tối ưu hóa thông lượng ghi:** Thiết kế lược đồ cơ sở dữ liệu NoSQL trên nền tảng Amazon DynamoDB được phân rã thành bốn bảng chức năng biệt lập phục vụ công tác truy vết chuyên sâu: RegisteredDevices, SecurityAlerts, FirewallLogs, và SystemLogs. Tối ưu hóa cấu trúc bảng bằng việc lựa chọn chính xác các khóa phân vùng (Partition Key) và khóa sắp xếp (Sort Key) dựa trên định dạng thời gian (Timestamp) để đáp ứng mô hình ứng dụng ghi cường độ cao (Write-heavy application). Đồng thời, thiết lập các chỉ mục thứ cấp toàn cục (Global Secondary Indexes - GSI) như chỉ mục realtimeFetch để phục vụ tác vụ quét dữ liệu log mới nhất theo thời gian thực mà không làm ảnh hưởng đến hiệu năng chung hoặc gây tốn kém chi phí do phải thực hiện các lệnh quét toàn bảng (Scan).

- **Thứ tư, xây dựng giao diện giám sát thời gian thực và đồng bộ hóa mạng phân phối nội**

dung: Xây dựng ứng dụng Web Single Page Application (SPA) hoàn chỉnh sử dụng thư viện ReactJS, cung cấp cho quản trị viên bảng điều khiển an ninh tập trung hiển thị trực quan thông tin log hệ thống và cảnh báo IDPS với cơ chế tự động nạp lại dữ liệu (Auto-refresh) định kỳ sau mỗi 1,5 giây. Mã nguồn Frontend được lưu trữ an toàn trong Amazon S3 bucket riêng tư với cấu hình chặn hoàn toàn truy cập công khai từ bên ngoài (Block Public Access). Toàn bộ giao diện hệ thống sẽ được phân phối thông qua mạng lưới viên Amazon CloudFront sử dụng cơ chế xác thực quyền truy cập nguồn gốc Origin Access Control (OAC), bắt buộc áp dụng giao thức mã hóa đường truyền toàn diện HTTPS (TLS 1.2+), triệt tiêu hoàn toàn các lỗi bảo mật trình duyệt về Mixed Content khi gọi API và bảo vệ tính toàn vẹn của dữ liệu trên đường truyền.

CHƯƠNG 2.

PLANNING (LẬP KẾ HOẠCH DỰ ÁN)

2.1 Yêu cầu Hệ thống (System Request)

Nhu cầu nghiệp vụ (Business Need): Doanh nghiệp cần giải pháp giám sát an ninh mạng tập trung có khả năng chịu tải cao, không tốn tài nguyên quản trị phần cứng, và bảo vệ an toàn bằng chứng số không bị tin tặc xóa bỏ sau xâm nhập.

Yêu cầu nghiệp vụ (Business Requirements): Hệ thống phải có Agent thu thập log tự động (RESTful API), cơ chế định danh thiết bị đa lớp (IP Whitelist & Token). Quản trị viên cần được cung cấp giao diện hiển thị cảnh báo thời gian thực và công cụ truy vết lịch sử sự cố đa chiều.

Giá trị mang lại (Business Value): Chuyển đổi hoàn toàn mô hình tài chính từ việc phải mua sắm phần cứng đắt đỏ sang chi trả theo nhu cầu thực tế. Cụ thể, hệ thống loại bỏ triệt để chi phí đầu tư hạ tầng ban đầu (**CapEx** - Capital Expenditure) do không cần trang bị máy chủ vật lý. Đồng thời, tối ưu hóa chi phí vận hành thường xuyên (**OpEx** - Operational Expenditure) nhờ mô hình Serverless chỉ tính phí khi có phát sinh lưu lượng log, không lãng phí tài nguyên chạy không tải. Về mặt vận hành, hệ thống giúp rút ngắn đáng kể thời gian trung bình để xử lý và phục hồi sự cố (**MTTR** - Mean Time To Recovery) thông qua luồng dữ liệu cảnh báo an ninh được giám sát tập trung theo thời gian thực.

Vấn đề đặc thù (Special Issues): Dữ liệu log chứa thông tin nhạy cảm. Bắt buộc mã hóa đường truyền TLS/SSL 100% và chặn đứng các truy cập ngoại lai trái phép ngay từ mạng viền (Edge Network).

2.2 Phân tích Tính Khả thi (Feasibility Analysis)

2.2.1 Khả thi về Kỹ thuật:

Hệ thống sử dụng hoàn toàn các dịch vụ quản lý sẵn (fully-managed) của AWS, giúp loại bỏ các công việc quản trị máy chủ vật lý hay hệ điều hành. Vấn đề bảo mật và phân quyền được giải quyết thông qua AWS IAM Role, resource policy. Hệ thống áp dụng nguyên tắc "đặc quyền tối thiểu" (Least Privilege), tách biệt quyền của các hàm Lambda ghi dữ liệu (chỉ có quyền PutItem/BatchWriteItem) và luồng quản trị (chỉ có quyền Query), giảm thiểu rủi ro nếu có một

thành phần bị tấn công.

2.2.2 Khả thi về Kinh tế (Economic Feasibility):

Kiến trúc Serverless loại bỏ hoàn toàn chi phí đầu tư phần cứng ban đầu (CAPEX) và nhân sự bảo trì máy chủ, chuyển đổi 100% sang chi phí vận hành (OPEX) thông qua mô hình *Pay-as-you-go*. Để minh chứng tính khả thi khi áp dụng vào môi trường doanh nghiệp thực tế, bài toán chi phí được tính toán dựa trên kịch bản một hệ thống SOC (Security Operations Center) quy mô trung bình: Doanh nghiệp có **100 máy trạm (Agent)**, mỗi máy trạm đóng gói và gửi 1 lô (batch 25 logs) mỗi 5 giây.

Tổng lưu lượng hàng tháng: 100 máy trạm sẽ tạo ra khoảng **51,8 triệu API requests**, tương đương với việc thu thập, xử lý và lưu trữ **1,29 tỷ sự kiện an ninh** (ước tính dung lượng khoảng 650 GB/tháng). Chi phí triển khai hệ thống được hoạch định cụ thể như sau:

- **Amazon S3 & CloudFront (Phân phối giao diện):** Giao diện Dashboard tĩnh có dung lượng rất nhỏ (< 50MB). Bảng thông tra về cho đội ngũ quản trị viên truy cập hàng ngày nằm hoàn toàn trong giới hạn *Free Tier* của CloudFront (miễn phí 1 TB truyền tải dữ liệu (Data Transfer Out) và 10 triệu HTTP/S requests mỗi tháng). → **Chi phí ước tính: 0 USD/tháng.**
- **Amazon Cognito (Quản lý định danh):** Dịch vụ miễn phí hoàn toàn cho 50.000 người dùng hoạt động hàng tháng (MAU). Kích thước đội ngũ chuyên gia phân tích an ninh (SOC Analysts) của doanh nghiệp thường không vượt qua ngưỡng này. → **Chi phí ước tính: 0 USD/tháng.**
- **Amazon API Gateway (Tiếp nhận lưu lượng):** Phí duy trì cổng API là không có. Với kiến trúc REST API, AWS tính phí 3,50 USD cho mỗi 1 triệu request. Với 51,8 triệu request/tháng từ các thiết bị đầu cuối: → **Chi phí ước tính: $51,8 \times 3,50 \approx 181$ USD/tháng.**
- **AWS Lambda (Tầng xử lý Compute):** Thay vì sử dụng *Provisioned Concurrency* rất đắt đỏ để giữ ấm hàm (warm-start), hệ thống áp dụng **chế độ SnapStart trên nền tảng runtime Python 3.14** (miễn phí). Tính năng nâng cao này tạo ra một bản sao (snapshot) của toàn bộ bộ nhớ và trạng thái khởi tạo (init phase) của mã nguồn. Nhờ SnapStart, hiện tượng khởi động lạnh (Cold Start) đặc trưng của Serverless bị triệt tiêu, đưa thời gian phản hồi xuống mức dưới 50 mili-giây mà không tốn phí duy trì tài nguyên. Với 51,8 triệu lượt gọi (mức phí 0,20 USD/1 triệu) và thời gian thực thi trung bình 50ms cho cấu hình bộ nhớ 256MB, tổng chi phí tính toán khá nhỏ. → **Chi phí ước tính: ≈ 21 USD/tháng.**
- **Amazon DynamoDB (Lưu trữ và Truy vấn):** Hệ thống sử dụng *On-Demand Capacity Mode*, không tốn phí duy trì năng lực phần cứng nhân rồi. Chi phí lưu trữ cho 650 GB dữ liệu là 162,5 USD (0,25 USD/GB). Sử dụng logic gom lô (Batching 25 logs/request) từ máy trạm, số lượng đơn vị ghi (WRU) tiêu thụ được tối ưu hóa. Cước phí ghi cho 1,29 tỷ bản

ghi rơi vào khoảng 1.612 USD (1,25 USD/1 triệu WRU). → **Chi phí ước tính: $\approx$ 1.775 USD/tháng.**

Tổng kết đánh giá tài chính (TCO - Total Cost of Ownership):

Tổng chi phí vận hành toàn bộ kiến trúc trên AWS cho doanh nghiệp ước tính rơi vào khoảng 1.977 USD/tháng để giám sát và lưu trữ **1,29 tỷ sự kiện an ninh** (tương đương 650 GB/tháng).

Để đánh giá khách quan tính cạnh tranh của hệ thống, chúng ta đối chiếu với biểu giá công khai của các giải pháp SIEM theo mô hình SaaS (Software-as-a-Service) phổ biến trên thị trường:

- **Datadog Log Management:** Theo biểu giá chuẩn, Datadog tính phí thu thập dữ liệu (Ingest) là 0,10 USD/GB và phí lưu trữ chỉ mục (Retain/Index) là 1,70 USD cho mỗi 1 triệu sự kiện (lưu trữ 15 ngày). Với kích bản 650 GB và 1,29 tỷ sự kiện, chi phí ước tính trên Datadog sẽ là 65 USD (Ingest) + 2.193 USD (Index) $\approx$ 2.258 USD/tháng.
- **Splunk Cloud:** Mức giá trung bình trên thị trường cho lưu lượng xử lý từ 20 - 25 GB/ngày (tương đương 650 GB/tháng) thường dao động trong khoảng 2.500 - 3.000 USD/tháng tùy thuộc vào gói cam kết và tính năng mở rộng.

Như vậy, việc ứng dụng kiến trúc Serverless trong đồ án cho thấy mức chi phí vận hành trực tiếp tiết kiệm hơn khoảng **15% - 30%** so với các giải pháp thương mại. Tuy nhiên, giá trị kinh tế cốt lõi của kiến trúc này không chỉ nằm ở bài toán chênh lệch giá, mà nằm ở **độ linh hoạt của dòng tiền**. Không giống như các nền tảng SIEM yêu cầu trả phí nền tảng cố định (Base fee) hoặc cam kết hợp đồng dài hạn, kiến trúc Serverless trên AWS cho phép hệ thống tự động co giãn chi phí về 0 (Scale-to-Zero) trong các chu kỳ thấp điểm không có lưu lượng, đồng thời giúp doanh nghiệp tránh được rủi ro phụ thuộc vào một nhà cung cấp công nghệ duy nhất (Vendor Lock-in). Như vậy đồ án này là một giải pháp kiến trúc hoàn toàn khả thi và có thể ứng dụng trong thương mại hóa thực tiễn.

2.2.3 Khả thi về Tổ chức:

Giao diện Web Dashboard cung cấp công cụ giám sát tập trung. Quản trị viên chỉ cần thao tác trên giao diện trực quan thay vì phải SSH vào từng máy chủ và dùng các câu lệnh CLI phức tạp để đọc file log.

2.3 Lựa chọn Phương pháp Phát triển (Development Methodology)

Dự án áp dụng **Mô hình Xoắn ốc (Spiral Model)** nhằm ưu tiên quản trị rủi ro (Risk Management). Quá trình phát triển lặp lại qua từng phân lớp kiến trúc (Database → Compute → Edge → UI), kết hợp kiểm thử bảo mật ngay sau mỗi vòng lặp để loại bỏ sớm các lỗ hổng thiết kế trước khi mở rộng.

CHƯƠNG 3.

SYSTEMS ANALYSIS (PHÂN TÍCH HỆ THỐNG VÀ YÊU CẦU)

3.1 Phân tích Yêu cầu Chức năng (Functional Requirements)

3.1.1 Phân hệ Quản trị & Xác thực (Admin Authentication)

- **FR-01 (Đăng nhập bắt buộc):** Tích hợp cổng xác thực danh tính thông qua Amazon Cognito để truy cập Dashboard.
- **FR-02 (Quản lý phiên làm việc):** Cấp phát và duy trì chuỗi JSON Web Token (JWT) trong HTTP Header để xác thực quyền gọi API.

3.1.2 Phân hệ Quản lý Thiết bị (Device Management)

- **FR-03 (Khai báo định danh):** Quản trị viên đăng ký máy trạm, hệ thống sinh ra Machine_Token và Machine_Secret duy nhất.
- **FR-04 (Cập nhật thông tin):** Cho phép thay đổi trạng thái ACTIVE/BLOCKED. Mã định danh bị khóa (Read-only) để đảm bảo toàn vẹn.
- **FR-05 (Xóa với độ trễ):** Tích hợp cơ chế xác nhận khóa nút bấm 10 giây trước khi cho phép xóa vĩnh viễn thiết bị khỏi hệ thống.

3.1.3 Phân hệ Tiếp nhận & Định tuyến Dữ liệu (Log Ingestion & Routing)

- **FR-06 (Thu thập đa luồng):** Agent rà quét song song auth, syslog và cảnh báo Snort/Suricata, phân loại và đóng gói JSON đẩy qua HTTPS.
- **FR-07 (Định tuyến tốc độ cao):** AWS Lambda xác thực Token/Secret, duyệt mảng JSON $O(N)$ để phân luồng và dùng BatchWriteItem ghi hàng loạt xuống DynamoDB.

3.1.4 Phân hệ Giao diện Giám sát & Điều tra (Web Dashboard)

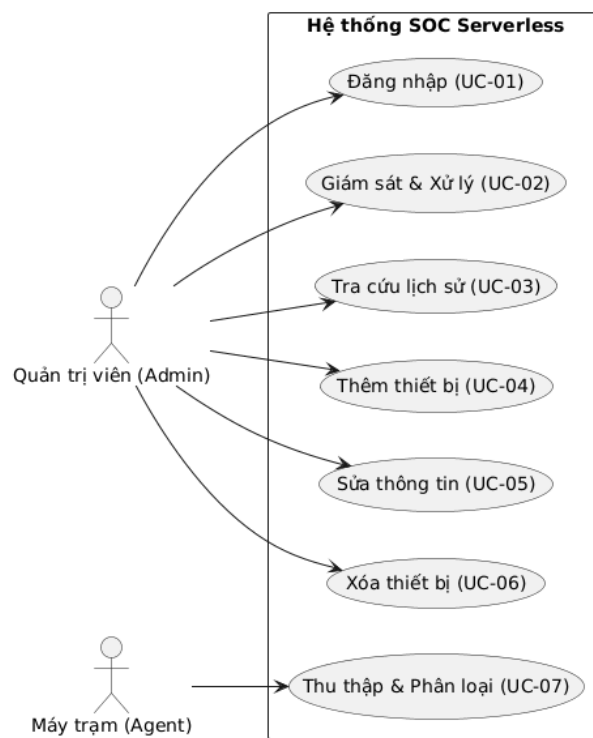
- **FR-08 (Real-time IDPS):** Giao diện chia 2 khu vực: luồng log hệ thống liên tục và danh sách cảnh báo tấn công ở trạng thái NEW.
- **FR-09 (Xử lý cảnh báo):** Hiển thị chi tiết log và thay đổi trạng thái sang RESOLVED/FALSE_POSITIVE.
- **FR-10 (Tra cứu lịch sử):** Bộ lọc đa tiêu chí linh hoạt, hiển thị đầy đủ thông tin phục vụ công

tác Forensics.

3.2 Phân tích Yêu cầu Phi Chức năng (Non-Functional Requirements)

- **NFR-01 (Bảo mật mạng viên):** API Gateway bắt buộc cấu hình IP Whitelist chặn toàn bộ truy cập lạ (HTTP 403) giúp chống DDoS tự động.
- **NFR-02 (Toàn vẹn dữ liệu):** Tầng Lambda xử lý bóc tách khóa bí mật để xác thực chống giả mạo thiết bị (Log Spoofing).
- **NFR-03 (Mã hóa đường truyền):** S3 Dashboard phân phối qua CloudFront HTTPS (TLS 1.2+). Mọi luồng Ingestion từ Agent cũng bị ép chuyển HTTPS.

3.3 Biểu đồ Ca sử dụng (Use Case Diagram)



Hình 3.1: Biểu đồ Use Case tổng quát của hệ thống

3.4 Đặc tả Ca sử dụng (Use Case Specification)

3.4.1 UC-01: Đăng nhập (Login)

UC-01: Đăng nhập (Login)	
Mã định danh	UC-01
Tên	Đăng nhập (Login)
Actor	Quản trị viên (Admin)
Mô tả	Admin truy cập giao diện Dashboard qua domain CloudFront.
Điều kiện tiên quyết	Admin đã có tài khoản hợp lệ được khởi tạo trong Cognito User Pool.
Điều kiện sau (Thành công)	Hệ thống lưu JWT vào Local Storage. Admin được điều hướng về giao diện Dashboard mặc định (chế độ Realtime).
Điều kiện sau (Thất bại)	Cognito trả về thông báo lỗi xác thực. Admin ở lại trang đăng nhập.
Luồng sự kiện chính	1. Admin truy cập URL CloudFront. 2. Hệ thống phát hiện chưa có JWT hợp lệ, tự động chuyển hướng đến Cognito Hosted UI. 3. Admin nhập tên đăng nhập và mật khẩu, nhấn <i>Sign In</i>. 4. Cognito xác thực thông tin, trả về Authorization Code qua URL redirect. 5. Frontend (react-oidc-context) dùng code để gọi API trao đổi token với Cognito. 6. Cognito cấp phát ID Token, Access Token và Refresh Token. 7. Frontend lưu token vào Local Storage; <code>api.js</code> tự động gắn <code>Bearer <ID Token></code> vào mọi request. 8. Hệ thống điều hướng Admin về trang Realtime Dashboard.
Luồng sự kiện thay thế	• [3a] Sai thông tin đăng nhập: Cognito trả về lỗi xác thực, hiển thị thông báo lỗi, Admin nhập lại từ bước 3. • [7a] Token hết hạn trong phiên làm việc: <code>api.js</code> nhận HTTP 401, tự động xóa Local Storage và yêu cầu đăng nhập lại.
Yêu cầu phi chức năng	Toàn bộ luồng giao tiếp bắt buộc qua HTTPS (TLS 1.2+) do S3 được bọc bởi CloudFront với OAC.

3.4.2 UC-02: Giám sát & Xử lý Cảnh báo Real-time

UC-02: Giám sát & Xử lý Cảnh báo Real-time	
Mã định danh	UC-02
Tên	Giám sát & Xử lý Cảnh báo Real-time
Actor	Quản trị viên (Admin)
Mô tả	Admin theo dõi log System, Firewall và danh sách cảnh báo được cập nhật liên tục trên Dashboard; thực hiện chuyển trạng thái cảnh báo từ NEW sang RESOLVED hoặc FALSE_POSITIVE.
Điều kiện tiên quyết	Admin đã đăng nhập thành công (UC-01) và JWT hợp lệ đang được lưu trong Local Storage.
Điều kiện sau (Thành công)	Trạng thái cảnh báo được cập nhật trong bảng SecurityAlerts; cảnh báo đã xử lý tự động biến mất khỏi danh sách NEW.
Điều kiện sau (Thất bại)	Trạng thái không thay đổi, hệ thống hiển thị thông báo lỗi.
Luồng sự kiện chính	- Hệ thống mở giao diện Realtime sau khi đăng nhập. - Frontend khởi động cơ chế Auto-refresh mỗi 1.500ms. - Frontend gọi GET /dashboard/realtime/{category} kèm JWT. - API Gateway xác thực JWT bằng Cognito Authorizer. - Lambda (QueryController) truy vấn các bảng log và bảng SecurityAlerts, lấy 50 bản ghi mới nhất. - Lambda ánh xạ device_id sang device_name và trả dữ liệu JSON. - Frontend cập nhật giao diện: panel trái hiển thị log, panel phải hiển thị các alert NEW. - Admin chọn một alert để xem chi tiết. - Admin nhấn <i>Xác nhận đã xử lý</i>; Frontend gọi PUT /alerts/status. - Lambda (AlertStatusUpdater) cập nhật alert_status = RESOLVED. - API trả HTTP 200; Frontend loại bỏ alert khỏi danh sách hiển thị.
Luồng sự kiện thay thế	[3a] JWT hết hạn: API Gateway trả HTTP 401; hệ thống xóa token và yêu cầu đăng nhập lại. [10a] Lỗi DynamoDB: Lambda trả HTTP 500; giao diện hiển thị lỗi và giữ nguyên trạng thái NEW.
Yêu cầu phi chức năng	Độ trễ cập nhật tối đa 1,5 giây. Lambda và DynamoDB On-Demand tự động mở rộng để xử lý tải cao.

3.4.3 UC-03: Tra cứu Lịch sử (Historical Inquiry)

UC-03: Tra cứu Lịch sử (Historical Inquiry)	
Mã định danh	UC-03
Tên	Tra cứu Lịch sử (Historical Inquiry)
Actor	Quản trị viên (Admin)
Mô tả	Admin chọn nguồn dữ liệu và các điều kiện lọc (thời gian, IP nguồn, loại tấn công, ...). Hệ thống truy vấn dữ liệu lịch sử qua GSI và hiển thị kết quả phục vụ điều tra sự cố.
Điều kiện tiên quyết	Admin đã đăng nhập thành công (UC-01).
Điều kiện sau (Thành công)	Danh sách bản ghi thỏa mãn điều kiện lọc được hiển thị dưới dạng bảng.
Điều kiện sau (Thất bại)	Hệ thống hiển thị thông báo “Không tìm thấy dữ liệu”.
Luồng sự kiện chính	1. Admin mở tab <i>Tra cứu Lịch sử</i>. 2. Admin chọn bảng dữ liệu mục tiêu (<code>table_target</code>). 3. Admin nhập các điều kiện lọc tùy chọn và khoảng thời gian tìm kiếm. 4. Admin nhấn <i>Tìm kiếm</i>. 5. Frontend gọi POST <code>/search</code> kèm JWT và payload truy vấn. 6. API Gateway xác thực JWT và kích hoạt Lambda. 7. Lambda xây dựng truy vấn trên GSI, thực thi tìm kiếm. 8. Kết quả được lọc bổ sung theo các trường trong <code>filters</code>. 9. Lambda trả dữ liệu JSON; Frontend hiển thị bảng kết quả.
Luồng sự kiện thay thế	• [3a] Không nhập bộ lọc: Hệ thống trả về các bản ghi mới nhất của bảng đã chọn. • [8a] Không có kết quả: Lambda trả mảng rỗng; Frontend hiển thị thông báo “Không tìm thấy dữ liệu phù hợp”.
Yêu cầu phi chức năng	Truy vấn phải sử dụng GSI thay vì Scan toàn bảng nhằm tối ưu hiệu năng và chi phí RCU.

3.4.4 UC-04: Thêm Thiết bị mới (Add Device)

UC-04: Thêm Thiết bị mới (Add Device)	
Mã định danh	UC-04
Tên	Thêm Thiết bị mới (Add Device)
Actor	Quản trị viên (Admin)
Mô tả	Admin đăng ký thiết bị mới. Hệ thống tự động sinh cặp định danh gồm Machine-Token và Machine-Secret, sau đó lưu thông tin vào bảng RegisteredDevices.
Điều kiện tiên quyết	Admin đã đăng nhập thành công (UC-01).
Điều kiện sau (Thành công)	Thiết bị mới được tạo trong bảng RegisteredDevices. Hệ thống hiển thị Machine-Token và Machine-Secret để Admin cấu hình Agent.
Điều kiện sau (Thất bại)	Không có bản ghi nào được tạo; hệ thống hiển thị thông báo lỗi.
Luồng sự kiện chính	- Admin mở tab <i>Quản lý Thiết bị</i> và chọn <i>Thêm thiết bị</i>. - Admin nhập device_name và trạng thái ban đầu. - Admin nhấn <i>Xác nhận</i>. - Frontend gọi POST /devices kèm JWT. - API Gateway xác thực JWT và kích hoạt Lambda. - Lambda sinh device_id và device_secret. - Lambda ghi bản ghi mới vào bảng RegisteredDevices. - API trả thông tin thiết bị vừa tạo. - Frontend hiển thị Machine-Token và Machine-Secret để Admin sao chép cấu hình cho Agent.
Luồng sự kiện thay thế	[7a] Lỗi DynamoDB: Hệ thống trả lỗi; thiết bị không được tạo.
Yêu cầu phi chức năng	Machine-Secret chỉ được hiển thị một lần sau khi tạo thiết bị và không thể truy xuất lại qua API.

3.4.5 UC-05: Sửa Thông tin Thiết bị (Edit Device Info)

UC-05: Sửa Thông tin Thiết bị (Edit Device Info)	
Mã định danh	UC-05
Tên	Sửa Thông tin Thiết bị (Edit Device Info)
Actor	Quản trị viên (Admin)
Mô tả	Admin chọn một thiết bị từ danh sách và chỉnh sửa các thuộc tính được phép (device_name, status). Các trường định danh cốt lõi (device_id, device_secret) không được phép thay đổi.
Điều kiện tiên quyết	Admin đã đăng nhập thành công (UC-01). Thiết bị cần chỉnh sửa đã tồn tại trong hệ thống.
Điều kiện sau (Thành công)	Thông tin thiết bị được cập nhật trong bảng RegisteredDevices. Nếu trạng thái chuyển sang BLOCKED, Agent của thiết bị sẽ bị từ chối xác thực từ lần gửi log tiếp theo.
Điều kiện sau (Thất bại)	Dữ liệu không thay đổi; hệ thống hiển thị thông báo lỗi.
Luồng sự kiện chính	- Admin mở tab <i>Quản lý Thiết bị</i>; hệ thống tải danh sách thiết bị qua GET /devices. - Admin chọn thiết bị cần chỉnh sửa và nhấn <i>Sửa</i>. - Hệ thống hiển thị form chỉnh sửa; các trường device_id và device_secret ở chế độ chỉ đọc. - Admin thay đổi device_name và/hoặc status. - Admin nhấn <i>Lưu</i>. - Frontend gọi PUT /devices kèm JWT và dữ liệu cập nhật. - API Gateway xác thực JWT và kích hoạt Lambda. - Lambda kiểm tra dữ liệu hợp lệ, cập nhật các trường được phép trong bảng RegisteredDevices. - Lambda trả HTTP 200; Frontend cập nhật lại danh sách thiết bị.
Luồng sự kiện thay thế	[8a] Payload chứa trường bị khóa: Lambda loại bỏ các trường không được phép cập nhật trước khi ghi dữ liệu. [8b] Không có trường hợp lệ: Hệ thống trả HTTP 400 và hiển thị thông báo lỗi.
Yêu cầu phi chức năng	Việc kiểm soát các trường được phép cập nhật phải được thực hiện tại Lambda, độc lập với Frontend, nhằm bảo đảm an toàn theo nguyên tắc Defense in Depth.

3.4.6 UC-06: Xóa Thiết bị (Delete Device)

UC-06: Xóa Thiết bị (Delete Device)	
Mã định danh	UC-06
Tên	Xóa Thiết bị (Delete Device)
Actor	Quản trị viên (Admin)
Mô tả	Admin yêu cầu xóa vĩnh viễn một thiết bị khỏi hệ thống. Hệ thống hiển thị hộp thoại xác nhận với bộ đếm ngược nhằm giảm nguy cơ thao tác nhầm.
Điều kiện tiên quyết	Admin đã đăng nhập thành công (UC-01). Thiết bị cần xóa đang tồn tại trong hệ thống.
Điều kiện sau (Thành công)	Thiết bị bị xóa khỏi bảng RegisteredDevices. Các yêu cầu gửi log từ Agent sử dụng thiết bị này sẽ không còn được chấp nhận.
Điều kiện sau (Thất bại / Hủy bỏ)	Không có thay đổi nào trong cơ sở dữ liệu.
Luồng sự kiện chính	- Admin nhấn nút <i>Xóa</i> trên thiết bị cần loại bỏ. - Hệ thống hiển thị hộp thoại xác nhận; nút <i>Xác nhận Xóa</i> bị vô hiệu hóa trong thời gian đếm ngược. - Bộ đếm ngược 10 giây được kích hoạt. - Sau khi đếm ngược kết thúc, nút <i>Xác nhận Xóa</i> được kích hoạt. - Admin nhấn <i>Xác nhận Xóa</i>. - Frontend gọi DELETE /devices kèm JWT và device_id. - API Gateway xác thực JWT và kích hoạt Lambda. - Lambda xóa thiết bị khỏi bảng RegisteredDevices. - Lambda trả HTTP 200; Frontend cập nhật lại danh sách thiết bị.
Luồng sự kiện thay thế	[3a] Admin hủy thao tác: Hộp thoại đóng lại và không có yêu cầu nào được gửi tới hệ thống. [8a] Lỗi cơ sở dữ liệu: Hệ thống trả lỗi; thiết bị vẫn được giữ nguyên trong danh sách.
Yêu cầu phi chức năng	Cơ chế xác nhận bằng bộ đếm ngược phải được áp dụng bắt buộc nhằm hạn chế rủi ro xóa nhầm thiết bị trong môi trường vận hành thực tế.

3.4.7 UC-07: Thu thập và Phân loại Log (Log Collection & Classification)

UC-07: Thu thập và Phân loại Log (Log Collection & Classification)	
Mã định danh	UC-07
Tên	Thu thập và Phân loại Log (Log Collection & Classification)
Actor	Máy trạm – Python Agent (Tác nhân tự động)
Mô tả	Agent tự động thu thập log từ máy trạm, phân loại dữ liệu, đóng gói theo batch và gửi lên hệ thống trung tâm để xác thực và lưu trữ.
Điều kiện tiên quyết	Agent đã được cấu hình đúng DEVICE_ID, DEVICE_SECRET và API_URL. Thiết bị có trạng thái ACTIVE và địa chỉ IP được phép truy cập hệ thống.
Điều kiện sau (Thành công)	Các bản ghi log được lưu vào đúng bảng DynamoDB. File .pos được cập nhật để xác nhận dữ liệu đã được ghi nhận thành công.
Điều kiện sau (Thất bại)	File .pos không được cập nhật; Agent giữ lại dữ liệu và tự động thử gửi lại. Không có log bị mất.
Luồng sự kiện chính	1. Agent khởi động và tạo các luồng đọc log song song. 2. Các luồng đọc theo dõi các file log hệ thống và đọc từ vị trí đã lưu trong file .pos. 3. Khi phát hiện log mới, Agent đóng gói dữ liệu và đưa vào hàng đợi xử lý. 4. Luồng gửi gom dữ liệu thành batch và chuẩn bị yêu cầu gửi. 5. Agent gửi batch log tới API Gateway kèm thông tin xác thực thiết bị. 6. API Gateway kiểm tra IP nguồn và chuyển tiếp yêu cầu hợp lệ tới Lambda. 7. Lambda xác thực thiết bị dựa trên Machine-Token và Machine-Secret. 8. Lambda phân loại log thành System Log, Firewall Log hoặc Security Alert. 9. Lambda ghi dữ liệu vào các bảng DynamoDB tương ứng. 10. Lambda trả HTTP 200; Agent cập nhật file .pos để xác nhận dữ liệu đã được lưu an toàn.

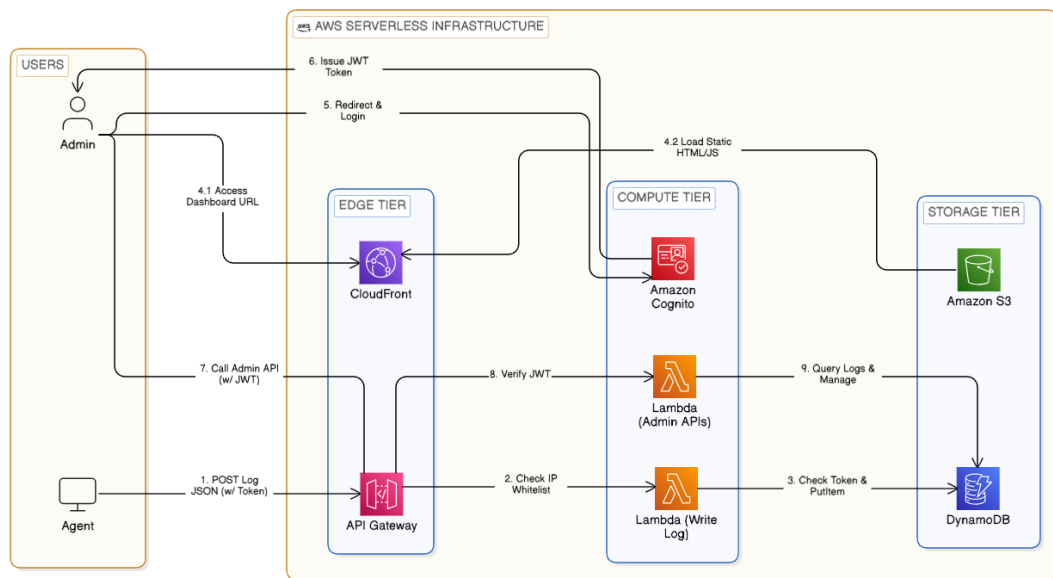
UC-07: Thu thập và Phân loại Log (tiếp)	
Luồng sự kiện thay thế	• [6a] IP không hợp lệ: API Gateway trả HTTP 403; Agent dừng gửi dữ liệu chờ xử lý. • [7a] Thiết bị không hợp lệ hoặc bị BLOCKED: Lambda trả HTTP 401; Agent dừng gửi dữ liệu và chờ được cấu hình lại. • [5a] Mất kết nối mạng hoặc timeout: Agent giữ dữ liệu trong hàng đợi và tự động thử lại cho đến khi kết nối được khôi phục hoặc dừng gửi và thông báo lỗi kết nối sau 90s. • [3a] Log Rotation: Agent phát hiện file log mới và tự động tiếp tục thu thập từ đầu file. • [Tắt máy]: Agent xử lý tín hiệu dừng, cố gắng gửi các batch còn lại trước khi thoát.
Yêu cầu phi chức năng	• Cơ chế ACK dựa trên file .pos phải bảo đảm không mất dữ liệu ngay cả khi Agent hoặc máy trạm gặp sự cố. • Quyền IAM của Lambda phải được giới hạn theo nguyên tắc đặc quyền tối thiểu (Least Privilege). • Hệ thống phải có cơ chế kiểm soát log trùng lặp nhằm tránh quá tải bộ nhớ và tài nguyên xử lý.

CHƯƠNG 4.

SYSTEMS DESIGN (THIẾT KẾ HỆ THỐNG)

Hệ thống áp dụng mô hình Serverless phân tán (Decoupled Serverless). Luồng tiếp nhận dữ liệu (Write Flow) và luồng quản trị (Auth & Read/Write Flow) được phân tách thành các hàm xử lý hoàn toàn độc lập nhằm đảm bảo cô lập lỗi (Fault Isolation) và tối ưu khả năng co giãn.

4.1 Kiến trúc Hệ thống Tổng thể (System Architecture)



Hình 4.1: Sơ đồ kiến trúc hạ tầng Serverless Logging phân tán

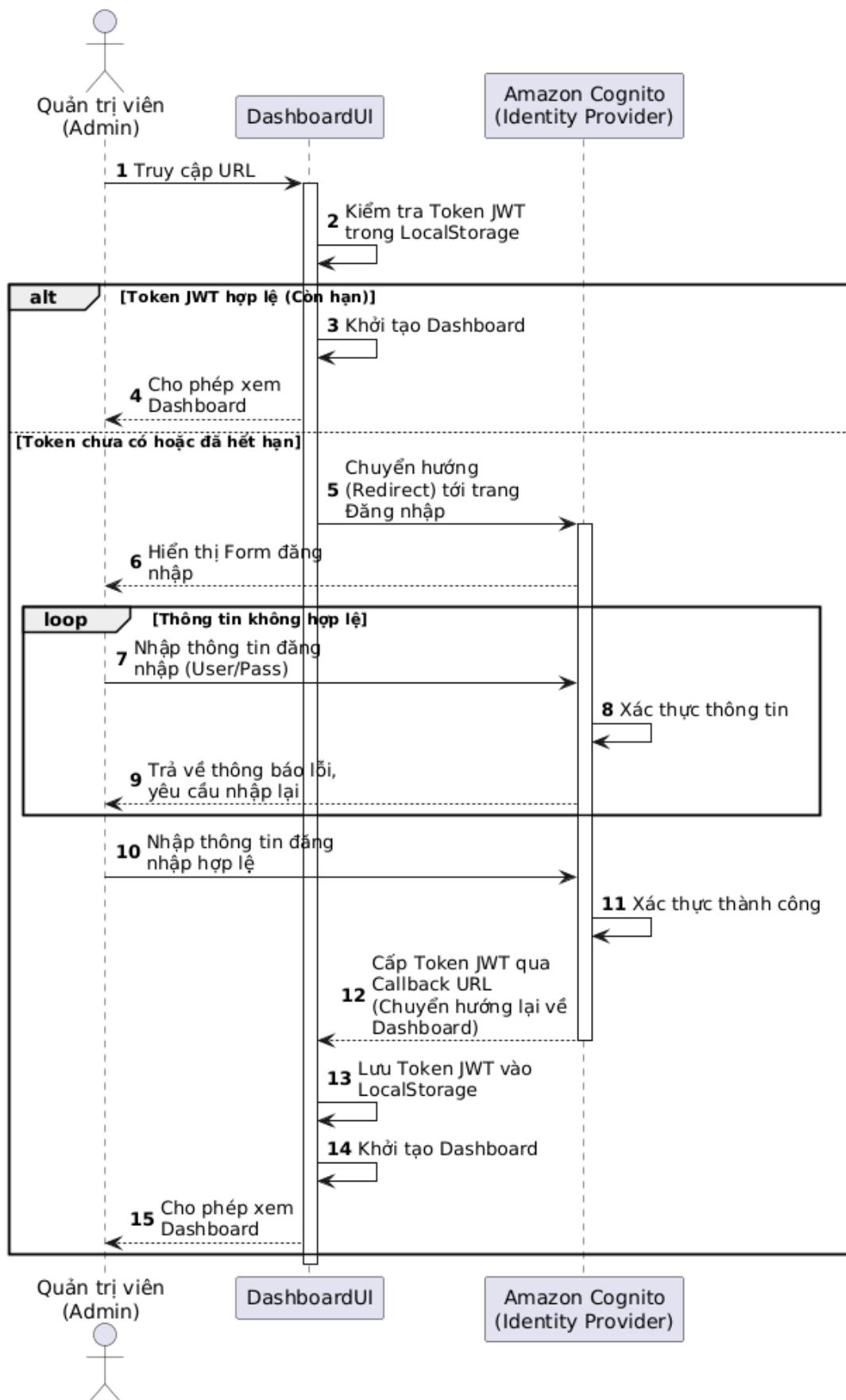
1. Luồng Thu thập Dữ liệu (Write Flow - Agent):

- **Bước 1 (Thu thập & Đóng gói):** Máy trạm thu thập sự kiện log hệ thống và cảnh báo IDS, đóng gói JSON đính kèm Machine_Token/Secret đẩy qua giao thức HTTPS.
- **Bước 2 (Kiểm duyệt dải IP):** API Gateway chặn toàn bộ IP nằm ngoài Whitelist của doanh nghiệp.
- **Bước 3 (Ghi dữ liệu):** AWS Lambda xác thực chéo thiết bị trong DynamoDB. Nếu ACTIVE, hàm sẽ dùng BatchWriteItem ghi hàng loạt dữ kiện vào các bảng log tương ứng.

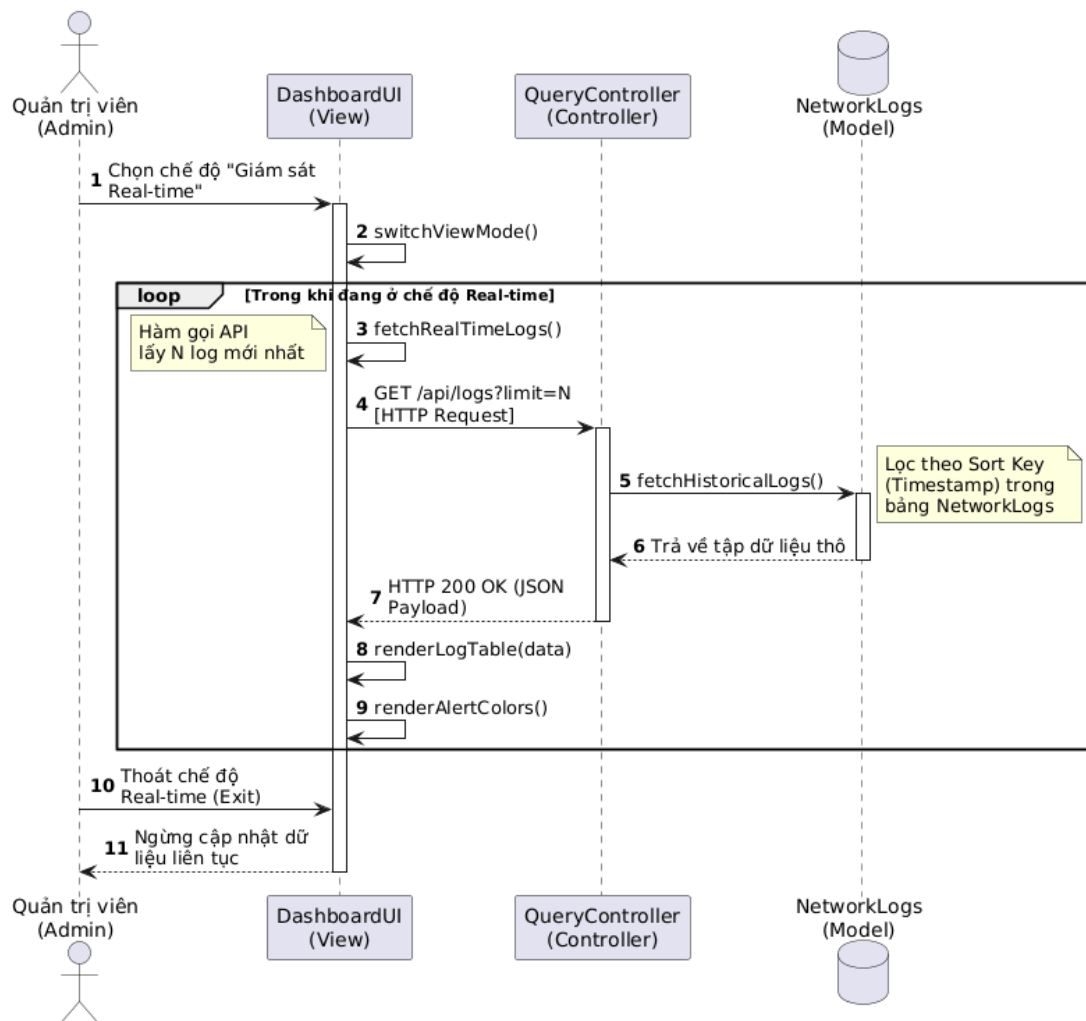
2. Luồng Quản trị và Giám sát (Auth & Read/Write Flow - Admin):

- **Bước 4 & 5 (Tải giao diện & Đăng nhập):** Trình duyệt tải mã nguồn ReactJS từ S3 thông qua CloudFront HTTPS. Nếu chưa đăng nhập, Admin được điều hướng qua Amazon Cognito.
- **Bước 6 (Cấp phát Token):** Đăng nhập thành công, Cognito trả về cụm JSON Web Token (ID, Access, Refresh).
- **Bước 7 & 8 (Xác thực API):** Admin gửi yêu cầu tra cứu key JWT. Cognito Authorizer tại API Gateway sẽ giải mã và kiểm chứng tính hợp lệ ngay ở lớp ngoài.
- **Bước 9 (Thực thi):** AWS Lambda (Admin APIs) tiếp nhận, truy vấn DynamoDB và trả về kết quả để hiển thị lên bảng điều khiển Dashboard.

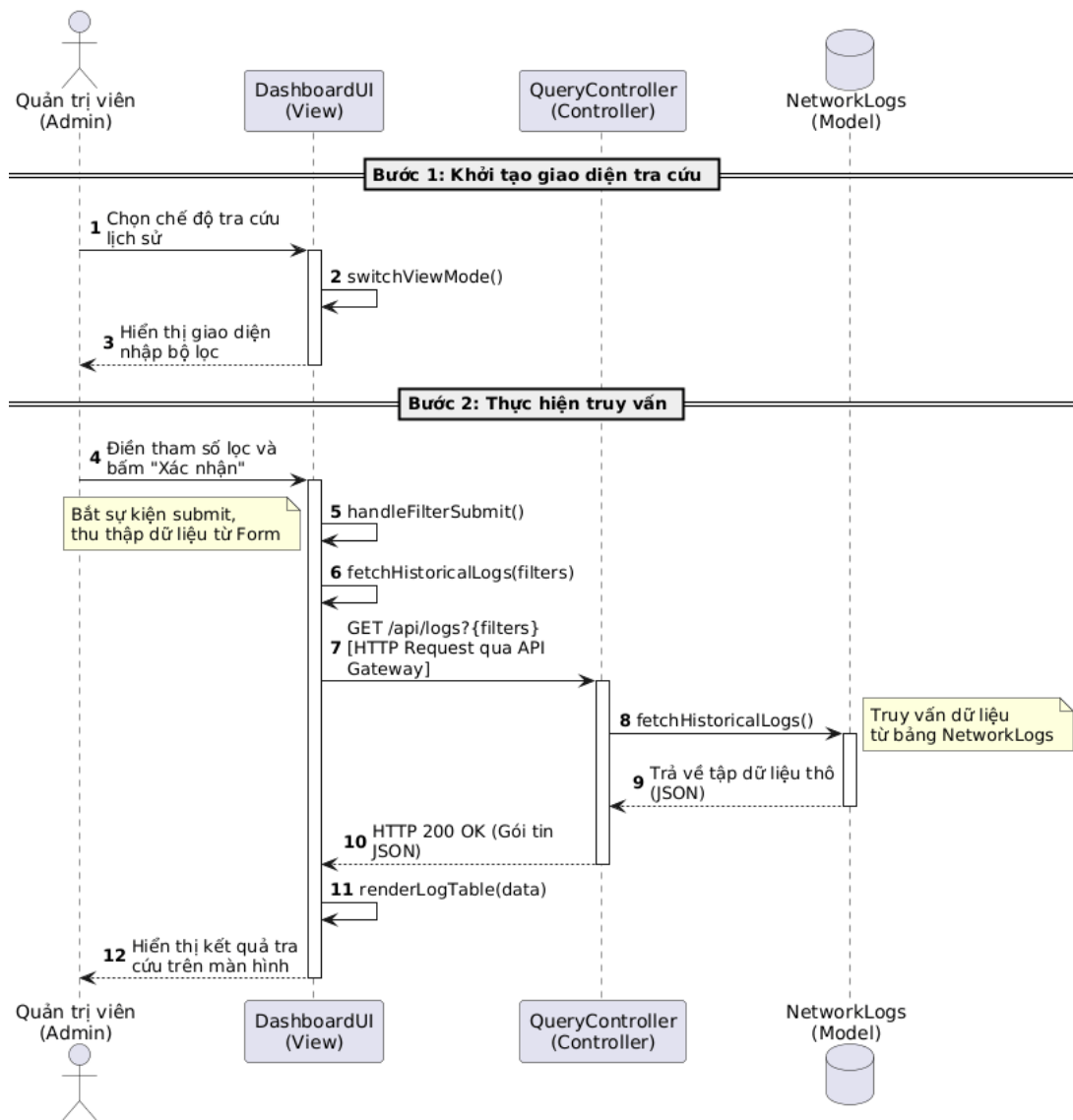
4.2 Biểu đồ Tuần tự (Sequence Diagram)



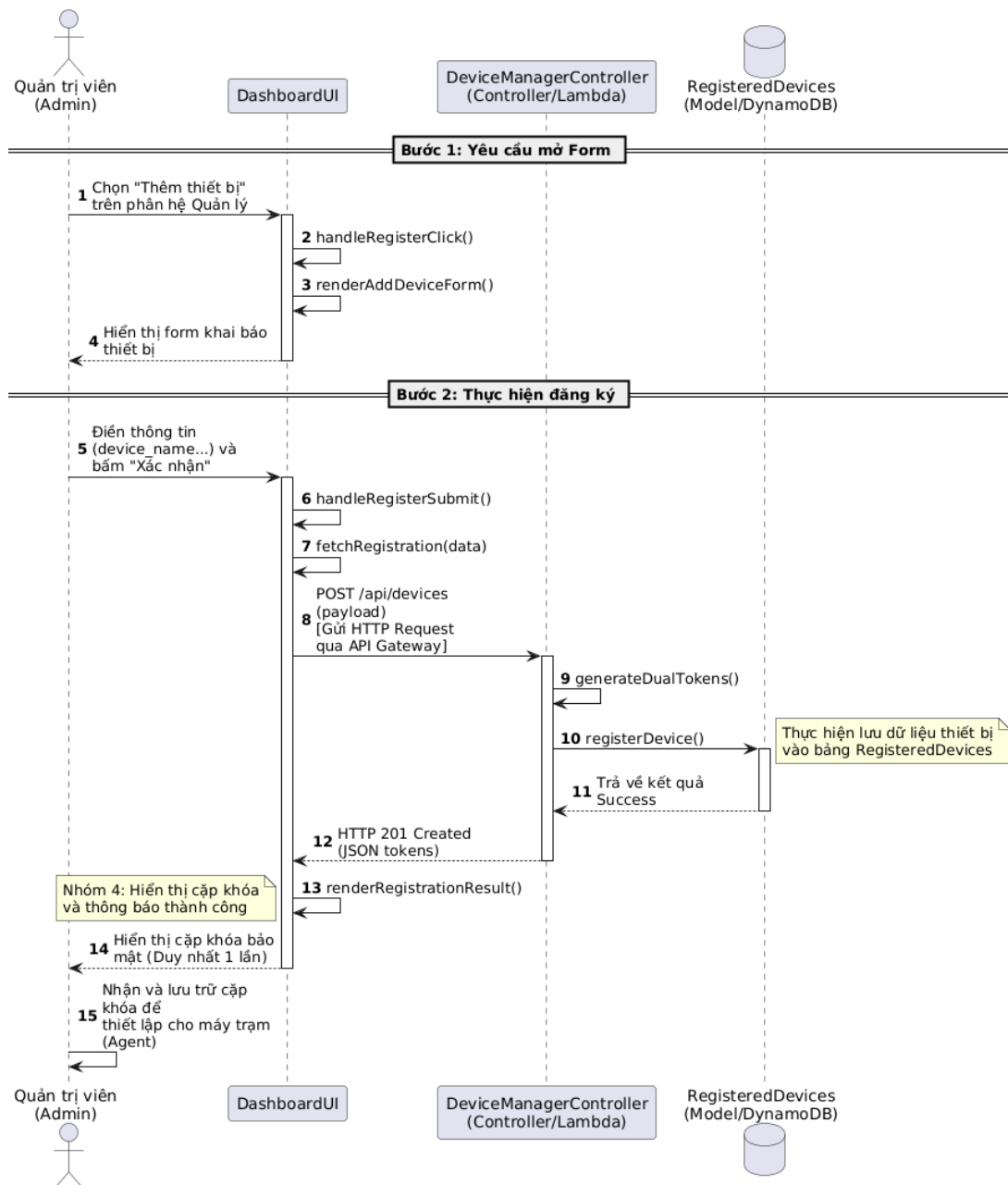
Hình 4.2: Biểu đồ tuần tự cho chức năng Đăng nhập Admin (UC-01)



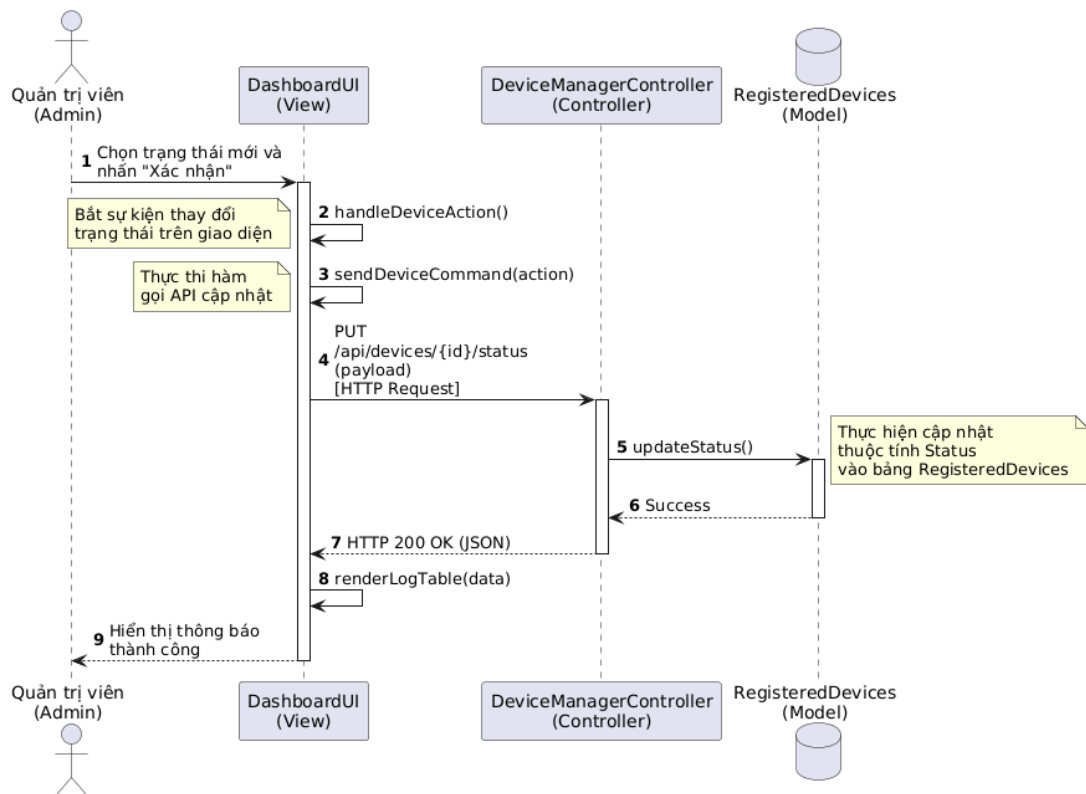
Hình 4.3: Biểu đồ tuần tự cho chức năng Giám sát & Xử lý cảnh báo Real-time (UC-02)



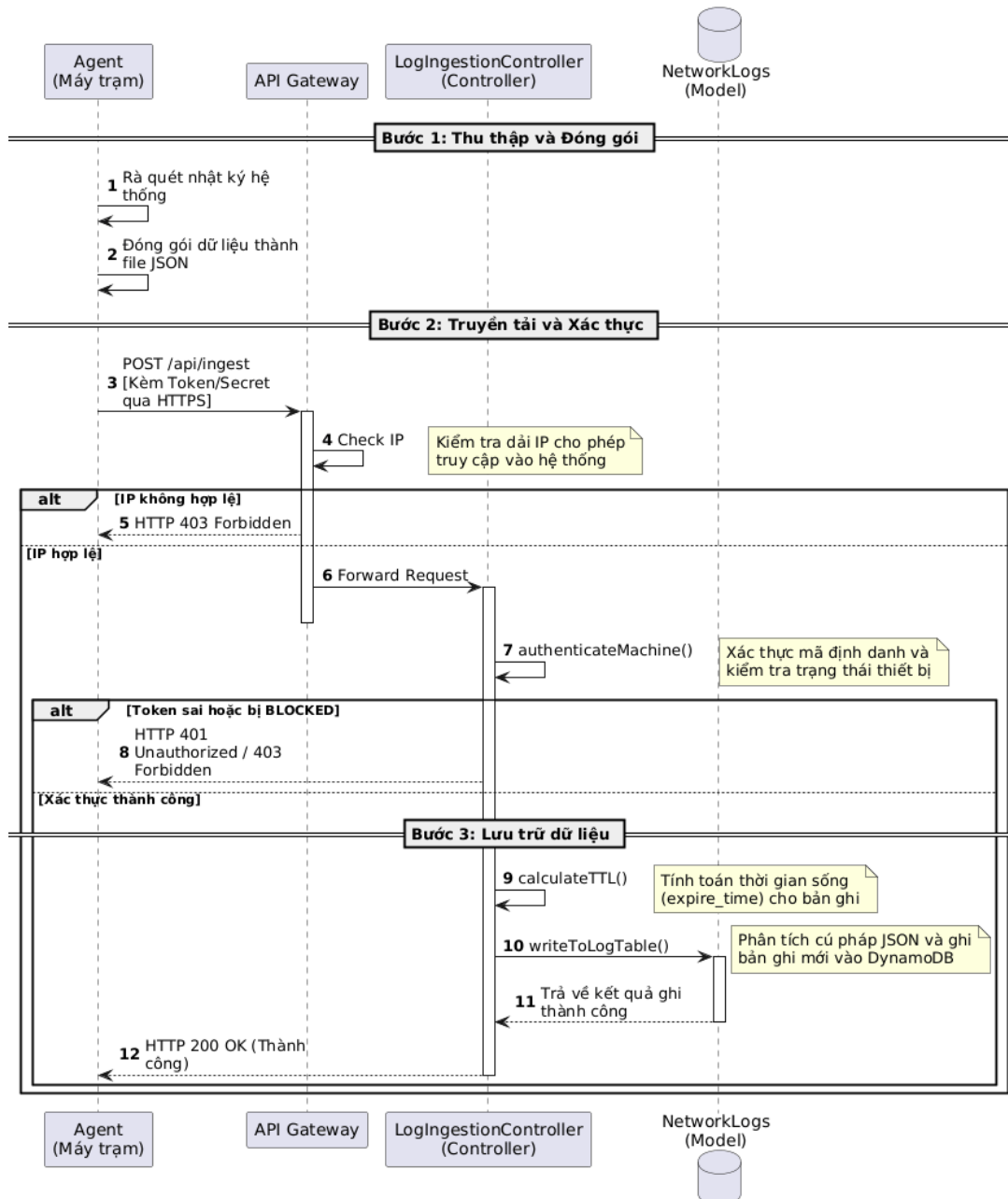
Hình 4.4: Biểu đồ tuần tự cho chức năng Tra cứu Lịch sử (UC-03)



Hình 4.5: Biểu đồ tuần tự cho chức năng Thêm thiết bị mới (UC-04)



Hình 4.6: Biểu đồ tuần tự cho chức năng Chỉnh sửa thông tin và Xóa thiết bị (UC-05/UC-06)



Hình 4.7: Biểu đồ tuần tự cho luồng Tự động thu thập và Phân loại log từ máy trạm (UC-07)

4.3 Các Cơ chế Bảo vệ của Hệ thống (Security Mechanisms)

Hệ thống được thiết kế với mô hình bảo mật nhiều lớp, đảm bảo an toàn từ khâu người dùng truy cập giao diện, truyền tải dữ liệu, cho đến tầng xử lý logic và lưu trữ cơ sở dữ liệu.

4.3.1 Bảo vệ lớp Frontend và Mạng truyền tải (S3 & CloudFront)

Nguy cơ bảo mật tiềm ẩn: Nếu giao diện tĩnh (ReactJS) được lưu trữ trên Amazon S3 và mở quyền truy cập công khai (Public Access), hệ thống sẽ đối mặt với rủi ro rò rỉ mã nguồn, cấu hình nhạy cảm, và các cuộc tấn công thay đổi nội dung (Defacement). Bên cạnh đó, việc truy cập S3

trực tiếp qua giao thức HTTP không được mã hóa tạo điều kiện cho tấn công Man-in-the-Middle (MitM). Đồng thời, trình duyệt web hiện đại sẽ chặn đứng các luồng gọi API nếu phát sinh lỗi *Mixed Content* (Nội dung hỗn hợp) — tức là một trang web HTTP cố gắng gọi đến API Gateway đang chạy HTTPS.

Cơ chế phòng ngự:

- **Khóa truy cập S3 và áp dụng OAC:** Tính năng chặn toàn bộ quyền truy cập công khai (*Block Public Access*) của S3 Bucket được kích hoạt. Điều này có nghĩa là mọi nỗ lực truy cập trực tiếp từ Internet vào URL gốc của S3 đều bị từ chối. Thay vào đó, hệ thống sử dụng **Origin Access Control (OAC)**. OAC tự động tạo các chữ ký điện tử (SigV4) trên mỗi yêu cầu từ CloudFront gửi về S3. Nhờ đó, S3 chỉ cấp phép đọc file cho duy nhất bản phân phối CloudFront hợp lệ, ép buộc lưu lượng phải đi qua lớp bảo mật HTTPS của CloudFront.
- **Mã hóa HTTPS từ biên:** Amazon CloudFront đóng vai trò là CDN trung gian, kết thúc SSL/TLS (HTTPS termination) ngay tại các điểm biên (Edge Locations). Điều này không chỉ mã hóa toàn bộ dữ liệu truyền tải mà còn giải quyết triệt để lỗi *Mixed Content*, cho phép mã JavaScript từ Frontend thực hiện các lệnh gọi API xác thực đến API Gateway một cách an toàn và hợp lệ.

4.3.2 Bảo vệ lớp API và Quản lý Lưu lượng (API Gateway)

Là cửa ngõ giao tiếp duy nhất giữa bên ngoài và hệ thống lõi, API Gateway được trang bị các cơ chế kiểm soát nghiêm ngặt dành riêng cho từng nhóm đối tượng (Admin và Agent).

a, Xác thực quyền quản trị (Cognito Authorizer)

Tin tặc có thể giả mạo các lệnh gọi HTTP POST/GET để truy cập trái phép vào các API quản trị, từ đó đánh cắp dữ liệu log hoặc xóa cấu hình thiết bị.

Các API dành cho Admin được gắn **Cognito Authorizer**. Khi Frontend gửi yêu cầu, nó phải đính kèm JSON Web Token (JWT) hợp lệ trên HTTP Header. API Gateway sẽ tự động kiểm tra tính hợp lệ, thời hạn và chữ ký của token này ngay tại tầng biên. Nếu token bị thiếu, sai, hoặc hết hạn, yêu cầu sẽ bị từ chối ngay lập tức (trả về HTTP 401 Unauthorized) mà không hề chạm đến tầng Lambda. Việc này giúp bảo vệ logic nghiệp vụ và tiết kiệm chi phí tính toán (compute cost) phát sinh cho các yêu cầu trái phép.

b, Bộ lọc IP cho luồng dữ liệu Log (IP Whitelisting)

Kẻ tấn công có thể phát hiện ra endpoint của API Log Ingestion và gửi hàng loạt log giả mạo (Log Spoofing/Data Poisoning) nhằm làm sai lệch dữ liệu cảnh báo, gây nhiễu loạn hệ thống giám sát.

Agent tại máy trạm không sử dụng xác thực người dùng (người thật), API Ingestion được bảo vệ bởi **Resource Policy** của API Gateway. Chính sách này định nghĩa một danh sách trắng

(Whitelist) chứa các địa chỉ IP công khai hợp lệ của các máy trạm/máy chủ đã đăng ký. Bất kỳ yêu cầu POST nào xuất phát từ một IP ngoài Whitelist sẽ bị API Gateway thả bỏ ngay lập tức với mã lỗi HTTP 403 Forbidden. Việc này đảm bảo chỉ các máy trạm nội bộ doanh nghiệp mới có quyền gửi logs lên hệ thống, tránh khả năng hacker tấn công từ ngoài.

b, Chống tấn công từ chối dịch vụ (DoS/DDoS Protection)

Tấn công DoS/DDoS hoặc bão log (Log Storms) có thể tạo ra lưu lượng yêu cầu khổng lồ trong thời gian ngắn. Việc này không chỉ có thể đánh sập hệ thống (exhaustion of concurrency) mà còn gây ra rủi ro về mặt kinh tế (Economic Denial of Sustainability - EDoS) do các dịch vụ như Lambda và DynamoDB tự động mở rộng và tính phí theo lưu lượng thực tế.

Hệ thống giải quyết bài toán này bằng cách kết hợp hai lớp kiểm soát lưu lượng độc lập của API Gateway là **Throttling** và **Usage Plans**:

- **Throttling (Điều tiết lưu lượng ở cấp độ Stage):** Hệ thống áp dụng cấu hình Throttling mặc định cho toàn bộ *Stage* triển khai. Thông qua thuật toán *Token Bucket*, cơ chế này thiết lập giới hạn tốc độ xử lý ổn định (Rate - Requests Per Second) và khả năng chịu tải bùng nổ tức thời (Burst) cho toàn bộ API. Lớp phòng vệ này đóng vai trò như một màng lọc vật lý, đảm bảo tổng lưu lượng đi vào hệ thống không bao giờ vượt quá ngưỡng chịu đựng an toàn của các dịch vụ Backend (Lambda, DynamoDB).
- **Usage Plans (Kiểm soát lưu lượng ở cấp độ Client):** Để tránh tình trạng một máy trạm (Agent) bị nhiễm mã độc và gửi yêu cầu spam làm cạn kiệt tài nguyên chung của toàn hệ thống, hệ thống áp dụng *Usage Plans*. Usage Plans cho phép định nghĩa **Quota** (Hạn mức sử dụng) – tức là tổng số lượng request tối đa mà một Client cụ thể được phép gọi trong một khoảng thời gian (ngày/tuần/tháng).

Khi có bất kỳ dấu hiệu bất thường nào làm lưu lượng vượt quá giới hạn Throttling của Stage, hoặc một Client dùng cạn kiệt Quota trong Usage Plan của mình, API Gateway sẽ chủ động thả bỏ (drop) các request dư thừa ngay tại tầng mạng biên và trả về mã lỗi HTTP 429 Too Many Requests. Cơ chế này đảm bảo hệ thống cốt lõi luôn hoạt động trong trạng thái ổn định và chi phí đám mây được kiểm soát chặt chẽ một cách tự động.

4.3.3 Bảo vệ lớp Tính toán và Dữ liệu (Lambda & DynamoDB)

Nếu mã nguồn Lambda chứa lỗ hổng bảo mật (chẳng hạn như lỗi chèn mã từ thư viện bên thứ ba), tin tặc có thể lợi dụng để leo thang đặc quyền (Privilege Escalation) và thực thi các lệnh phá hoại như xóa toàn bộ bảng cơ sở dữ liệu DynamoDB hoặc chỉnh sửa cấu hình hệ thống trái phép. Để vô hiệu hóa hoàn toàn rủi ro này, kiến trúc bảo mật nội bộ được thiết kế xoay quanh hai nguyên tắc chính:

- **Thiết lập Identity-based IAM Role với Đặc quyền tối thiểu (Least Privilege):** Không hàm

Tối ưu hóa quản lý phân quyền tập trung: Mặc dù AWS DynamoDB đã hỗ trợ *Resource-based Policy*, phương pháp này không được áp dụng trực tiếp trên từng bảng cơ sở dữ liệu để kiểm soát quyền của Lambda. Việc phân quyền được tập trung quản lý hoàn toàn thông qua Identity-based IAM Role gắn trên Lambda nhằm đảm bảo tính tập trung và dễ bảo trì. Nếu sử dụng Resource Policy trên cơ sở dữ liệu, mọi thay đổi về mức quyền của Lambda sẽ kéo theo yêu cầu phải rà soát và chỉnh sửa chính sách tài nguyên của hàng loạt bảng khác nhau, làm tăng độ phức tạp vận hành. Hơn nữa, *Resource Policy* của DynamoDB được khuyến nghị ưu tiên cho các kịch bản tích hợp chia sẻ dữ liệu liên tài khoản (Cross-Account Access), thay vì sử dụng cho các vi dịch vụ (microservices) nội bộ.

- **Phân tách kiến trúc (Decoupling):** Chuyển đổi luồng xử lý từ đồng bộ (Synchronous) sang bất đồng bộ (Asynchronous).
- **Hấp thụ tải đột biến (Buffering):** Hứng chịu các đợt bão log (log storms) để bảo vệ cơ sở dữ liệu phía sau không bị quá tải.

4.4.2 Đánh giá tính phù hợp và Bài toán chi phí

Mặc dù SQS là một giải pháp tiêu chuẩn và tốt cho vấn đề hấp thụ tải đột biến, nhưng việc áp dụng nó vào kiến trúc hiện tại có phần không cần thiết (Overengineering):

- **Về khả năng chống chịu bão log:** Vấn đề tải đột biến đã được hệ thống giải quyết thông qua sự kết hợp của API Gateway và DynamoDB On-Demand. Khi lưu lượng vượt ngưỡng an toàn, API Gateway sẽ tự động giới hạn tốc độ (Throttling). Các lô log tạm thời bị từ chối sẽ được máy trạm (Agent) đưa vào queue nội bộ và tự động gửi lại ngay sau đó. Cơ chế này đảm bảo không mất log và hoàn toàn phù hợp với khả năng xử lý của Backend mà không cần thêm lớp bộ đệm trung gian.
- **Về bài toán chi phí:** Việc tận dụng Agent tại máy trạm để chờ và gửi lại log là giải pháp không phát sinh chi phí. Nếu sử dụng SQS, hệ thống sẽ phải gánh thêm cước phí của dịch vụ này (mức giá tiêu chuẩn là 0,40 USD cho mỗi 1 triệu request). Với đặc thù của hệ thống giám sát gửi log liên tục 24/7, khoản phí này sẽ cộng dồn đáng kể mà không giải quyết vượt trội vấn đề về tải so với kiến trúc hiện tại.
- **Về độ trễ thời gian thực (Latency):** Tính năng quản lý log theo thời gian thực (realtime dashboard) của hệ thống yêu cầu tốc độ phản hồi thấp. Nếu luồng dữ liệu đi qua SQS, AWS Lambda phải sử dụng một tiến trình quét ngầm (Event Source Mapping Poller) để liên tục kéo (poll) dữ liệu từ hàng đợi. Khi có bão log, tiến trình Poller này cần thời gian để khởi động và mở rộng số luồng quét. Quá trình này có thể khiến dữ liệu bị kẹt lại trong hàng đợi một thời gian, tăng độ trễ tổng thể.
- **Về độ phức tạp vận hành:** Việc bổ sung SQS đòi hỏi quản trị viên phải thiết lập thêm các cơ chế giám sát hàng đợi và cấu hình hàng đợi thư chết (Dead Letter Queue - DLQ) để giám giữ các lô dữ liệu lỗi. Điều này làm tăng độ phức tạp trong việc triển khai và bảo trì.

Kiến trúc luồng xử lý Đồng bộ trực tiếp (API Gateway → Lambda → DynamoDB) hiện tại là sự lựa chọn cân bằng hiệu quả ba yếu tố: tối ưu chi phí, độ trễ thời gian thực, và khả năng kiểm soát tải. Giải pháp Message Queue bằng SQS sẽ chuyển việc xử lý các vấn đề khi lưu logs chưa thành công lên hệ thống cloud hoàn toàn nhưng hiện tại các máy trạm đã có thể xử lý tốt mà không cần đến SQS. Việc kết hợp thêm SQS là không quá cần thiết về mặt hiệu năng và còn có thể làm gia tăng độ trễ tổng thể, chưa thực sự phù hợp với bài toán Realtime logs. Trong tương lai, định hướng nâng cấp hệ thống sang các dịch vụ truyền phát sự kiện (Event Streaming) như Amazon

Kinesis sẽ là lựa chọn phù hợp hơn để xử lý tải khổng lồ mà vẫn đảm bảo được tốc độ phản hồi tức thời.

CHƯƠNG 5.

TRIỂN KHAI VÀ CẤU HÌNH HỆ THỐNG (SYSTEM IMPLEMENTATION)

5.1 Cấu hình Agent tại máy trạm (Edge Endpoint Configuration)

Thành phần Agent tại máy trạm đóng vai trò là điểm tiếp nhận dữ liệu đầu cuối (Edge Data Ingestion). Kịch bản Agent được phát triển bằng ngôn ngữ Python, ứng dụng kiến trúc đa luồng (Multi-threading) để theo dõi và xử lý đồng thời 4 tệp nhật ký hệ thống: `/var/log/auth.log`, `/var/log/syslog`, `/var/log/ufw.log` (Nhật ký tường lửa), và `/var/log/suricata/fast.log` (Cảnh báo an ninh).

5.1.1 Lọc và Trích xuất dữ liệu tại nguồn (Edge-Filtering)

Agent thực hiện tiền xử lý, trích xuất ngữ cảnh an ninh và chuẩn hóa cấu trúc dữ liệu ngay tại bộ nhớ RAM của máy trạm trước khi truyền tải:

- **Trích xuất thông tin bằng Biểu thức chính quy (Regex):** Dữ liệu nhật ký thô từ các dịch vụ được phân tích cấu trúc thông qua các bộ mẫu Regex chuyên biệt được thiết lập sẵn. Hệ thống thực hiện bóc tách toàn bộ các trường thông tin trọng yếu bao gồm: *Source IP, Destination IP, Port, Protocol, Timestamp* đối với luồng tường lửa UFW; *Attack Type, Severity, Signature ID* đối với luồng cảnh báo Suricata; và *Event Type, User, Session Status* đối với luồng hệ thống (`auth.log/syslog`).
- **Chuẩn hóa và đồng bộ cấu trúc dữ liệu (Data Normalization):** Quá trình phân tách bằng Regex đồng thời đóng vai trò là màng lọc kiểm tra và xác thực tính toàn vẹn cấu trúc của chuỗi ký tự đầu vào. Các dòng dữ liệu không tương thích hoặc không chứa các thuộc tính sự kiện an ninh liên quan sẽ được phân loại để loại bỏ các thành phần nhiễu hệ thống, đảm bảo các bản ghi sau khi tiền xử lý đều được đóng gói thành định dạng cấu trúc JSON đồng nhất, bảo đảm tính tương thích tuyệt đối của cấu trúc payload đối với lược đồ (schema) lưu trữ phía sau, tránh các lỗi xung đột định dạng tại cổng biên AWS API Gateway.
- **Khuyến nghị tối ưu hóa:** Tùy thuộc vào chính sách bảo mật thực tế của doanh nghiệp, quản trị viên có thể cấu hình thêm giới hạn ngưỡng cảnh báo (`threshold / event_filter`) trực

tiếp trên hệ thống Suricata. Việc nén các cảnh báo trùng lặp giúp triệt tiêu hiện tượng nhiễu cảnh báo (Alert Fatigue) từ hệ điều hành, qua đó giảm đáng kể khối lượng dữ liệu thô mà Agent phải xử lý.

5.1.2 Đóng gói dữ liệu (Batching)

Để tối ưu hóa số lượng kết nối mạng (TCP Handshake) và giảm thiểu chi phí tính phí theo request của API Gateway, dữ liệu được gom thành các lô (Batch). Agent sẽ kích hoạt lệnh POST dữ liệu lên đám mây khi thỏa mãn một trong hai điều kiện: lô dữ liệu đạt đủ **25 bản ghi** (khớp với giới hạn của lệnh BatchWriteItem trên DynamoDB), hoặc sau **chu kỳ 5 giây**.

5.1.3 Cơ chế Xác nhận (ACK) và Quản lý tọa độ đọc

Để giải quyết bài toán chống mất mát dữ liệu (Zero Data Loss) và chống gửi lặp, Agent sử dụng một cơ chế đánh dấu trạng thái thông qua các tệp .pos (Position).

- Trong quá trình đọc file log, con trỏ tệp (byte offset) liên tục tịnh tiến. Tuy nhiên, mức tọa độ mới này chỉ được lưu trữ tạm thời trong bộ nhớ.
- Khi lô dữ liệu được gửi đi, Agent chờ phản hồi từ máy chủ. Tọa độ thực tế bên trong tệp .pos **chỉ được ghi đè** khi và chỉ khi Agent nhận được mã HTTP 200 OK từ AWS API Gateway.
- Nếu máy trạm bị sập nguồn đột ngột, tệp .pos vẫn giữ nguyên tọa độ của lần gửi thành công trước đó, đảm bảo dữ liệu chưa được Cloud xác nhận sẽ được đọc lại khi máy tính khởi động.

5.1.4 Xử lý Lỗi (Error Handling) và Thoát an toàn

Agent được lập trình để phân loại và xử lý mã lỗi HTTP:

- **Lỗi máy chủ (HTTP 5xx) hoặc đứt kết nối mạng:** Agent ghi nhận sự cố, bảo lưu lô dữ liệu hiện tại, tạm ngủ (sleep) và thực hiện gửi lại (Retry) cho đến khi thành công. Đồng thời kích hoạt cảnh báo Desktop Notification nội bộ nếu mất kết nối quá thời gian quy định (90 giây).
- **Lỗi xác thực (HTTP 401/403):** Nếu máy trạm bị chặn IP hoặc sai Machine-Token, Agent chủ động dừng tiến trình gửi nhằm tiết kiệm tài nguyên, và hiện thông báo để quản trị viên khắc phục sự cố.
- **Thoát an toàn (Graceful Shutdown):** Khi quản trị viên tắt Agent hoặc máy chủ khởi động lại (gửi tín hiệu SIGTERM/SIGINT), bộ xử lý ngắt (signal handler) sẽ can thiệp. Các luồng đọc sẽ tự động dừng, hệ thống dành thêm 5 giây để đóng gói và gửi nốt các log còn tồn đọng trong hàng đợi, cuối cùng cập nhật tệp .pos trước khi kết thúc hoàn toàn tiến trình.

5.2 Triển khai Hạ tầng Lưu trữ (Storage Tier)

Hạ tầng lưu trữ của hệ thống được quy hoạch độc lập thành hai thành phần cốt lõi: Tầng lưu trữ dữ liệu nhật ký động sử dụng cơ sở dữ liệu NoSQL Amazon DynamoDB và Tầng lưu trữ mã nguồn giao diện tĩnh sử dụng Amazon S3.

5.2.1 Cấu hình Cơ sở dữ liệu Amazon DynamoDB

Để đáp ứng khả năng mở rộng quy mô tuyến tính và hấp thụ lưu lượng bản ghi lớn từ nhiều máy trạm đồng thời, hệ thống triển khai 4 bảng cơ sở dữ liệu độc lập. Toàn bộ các bảng đều được cấu hình ở chế độ PAY_PER_REQUEST (On-Demand Capacity Mode) để tự động co giãn thông lượng ghi/đọc mà không gây thất cổ chai.

1. Bảng RegisteredDevices

Bảng lưu trữ thông tin định danh và quản lý trạng thái cấp phép truy cập của các máy trạm (Agent).

- **Key Schema:** device_id (Partition Key).

Tên thuộc tính	Kiểu dữ liệu	Mô tả chức năng
device_id	String	Mã định danh duy nhất (UUID) của thiết bị, đóng vai trò Khóa phân vùng.
device_name	String	Tên gọi nhớ của máy trạm để hiển thị trên giao diện quản lý.
device_secret	String	Chuỗi khóa bí mật dùng để thiết lập chữ ký xác thực (Authentication Signature) cho các lô dữ liệu tải lên.
status	String	Trạng thái cấp phép của thiết bị (ACTIVE cho phép gửi log, BLOCKED để ngắt kết nối).

Bảng 5.1: Cấu trúc thuộc tính bảng RegisteredDevices

2. Bảng SystemLogs

Bảng lưu trữ dữ liệu nhật ký hệ thống (auth.log, syslog) phục vụ công tác điều tra số (Forensics).

- **Key Schema:** device_id (Partition Key) và timestamp (Sort Key).
- **Chỉ mục thứ cấp (GSI):**
 - **DaemonIndex:** PK là daemon_name, SK là timestamp. Hỗ trợ truy vấn độc lập theo từng tiến trình.
 - **realtimeFetch:** PK là log_type, SK là timestamp. Phục vụ API lấy log mới nhất theo thời gian thực.

Tên thuộc tính	Kiểu dữ liệu	Mô tả chức năng
device_id	String	Khóa phân vùng, dùng để nhóm toàn bộ log của một máy trạm.
timestamp	String	Khóa sắp xếp, lưu thời gian phát sinh sự kiện theo chuẩn ISO 8601.
daemon_name	String	Tên tiến trình hệ thống sinh ra nhật ký (ví dụ: sshd, sudo).
log_type	String	Phân loại dữ liệu (Giá trị tĩnh: SYS), làm PK cho chỉ mục realtime.
raw_message	String	Toàn bộ chuỗi log gốc để rà soát nguyên bản khi cần thiết.
expire_time	Number	Mốc thời gian (Unix Epoch) để DynamoDB kích hoạt TTL, tự động xóa log cũ.

Bảng 5.2: Cấu trúc thuộc tính bảng SystemLogs

3. Bảng FirewallLogs

Bảng lưu trữ tập trung các sự kiện tường lửa thu thập từ UFW để phân tích lưu lượng mạng.

- **Key Schema:** device_id (Partition Key) và timestamp (Sort Key).
- **Chỉ mục thứ cấp (GSI):**
 - **SourceIpIndex:** PK là src_ip, SK là timestamp. Hỗ trợ truy vết ngược hành vi theo IP nguồn.
 - **realtimeFetch:** PK là log_type, SK là timestamp.

Tên thuộc tính	Kiểu dữ liệu	Mô tả chức năng
device_id	String	Khóa phân vùng.
timestamp	String	Khóa sắp xếp định tuyến thời gian.
action	String	Quyết định của tường lửa (BLOCK, ALLOW, AUDIT).
src_ip / dst_ip	String	Cặp địa chỉ IP Nguồn và Đích của gói tin mạng.
src_port/dst_port	String	Cổng dịch vụ Nguồn và Đích.
protocol	String	Giao thức truyền tải (TCP, UDP, ICMP).
log_type	String	Giá trị tĩnh: UFW. Dùng làm PK cho chỉ mục realtime.
raw_message	String	Toàn bộ chuỗi log UFW gốc.
expire_time	Number	Cấu hình TTL tự động dọn dẹp dữ liệu lưu lượng cũ.

Bảng 5.3: Cấu trúc thuộc tính bảng FirewallLogs

4. Bảng SecurityAlerts

Bảng lưu trữ các cảnh báo an ninh bóc tách từ hệ thống IDS Suricata.

- **Key Schema:** alert_id (Partition Key) và timestamp (Sort Key).
- **Chỉ mục thứ cấp (GSI):**
 - **StatusIndex:** PK là alert_status, SK là timestamp. Lọc nhanh cảnh báo chưa xử lý.
 - **AttackTypeIndex:** PK là attack_type, SK là timestamp. Phục vụ biểu đồ thống kê.
 - **realtimeFetch:** PK là log_type, SK là timestamp.

Tên thuộc tính	Kiểu dữ liệu	Mô tả chức năng
alert_id	String	Khóa phân vùng (UUID tự sinh) đại diện cho một sự kiện cảnh báo.
timestamp	String	Thời gian hệ thống ghi nhận tấn công (Sort Key).
device_id	String	Mã thiết bị chịu ảnh hưởng bởi cảnh báo.
attack_type	String	Loại hình tấn công trích xuất từ cấu trúc luật Suricata.
severity_level	String	Mức độ ưu tiên/nghiêm trọng (CRITICAL, HIGH, MEDIUM).
alert_status	String	Trạng thái xử lý sự cố (Ghi nhận ban đầu: NEW, Sau khi xử lý: RESOLVED).
log_type	String	Giá trị tĩnh: ALERT. Định tuyến cho luồng realtime.
raw_message_ref	String	Chuỗi cảnh báo thô để đối chiếu thông tin chi tiết.
expire_time	Number	Mốc thời gian kích hoạt TTL dọn dẹp dữ liệu.

Bảng 5.4: Cấu trúc thuộc tính bảng SecurityAlerts

5.2.2 Cấu hình Hạ tầng Khởi tạo Amazon S3 Bucket

Đối với luồng quản trị (Admin Dashboard), mã nguồn biên dịch tĩnh của ứng dụng Frontend (ReactJS) được lưu trữ tập trung tại một S3 Bucket chuyên biệt. Quy trình cấu hình phải tuân thủ các tiêu chuẩn bảo mật đám mây:

- **Kích hoạt Khóa truy cập công khai (S3 Block Public Access):** Tính năng *Block Public Access* được bật thành công ở trạng thái True cho toàn bộ cấu hình của Bucket. Cơ chế này ngăn chặn mọi hành vi truy cập trực tiếp từ Internet thông qua URL mặc định của Amazon S3 (s3.amazonaws.com) gây rủi ro lộ cấu hình mã nguồn client.
- **Cấu hình Ủy quyền Mạng biên qua OAC (Origin Access Control):** Để giao diện Dashboard hiển thị được trên trình duyệt của quản trị viên mà không cần mở công khai S3 Bucket, hệ thống tích hợp cơ chế ủy quyền thông qua Amazon CloudFront CDN:

- Khởi tạo một chính sách Origin Access Control (OAC) trên phân phối CloudFront, yêu cầu mọi kết nối ngược về S3 phải được ký số tự động theo chuẩn AWS SigV4.
- Cấu hình chính sách kiểm soát truy cập (Bucket Policy): Để đảm bảo an toàn tuyệt đối cho luồng giao tiếp, S3 Bucket được thiết lập một chính sách truy cập với hai điều kiện xác thực liên kết chặt chẽ. Cụ thể, S3 chỉ chấp nhận lệnh đọc tệp (s3:GetObject) nếu luồng truy cập xuất phát từ dịch vụ CloudFront (Service: cloudfront.amazonaws.com). Đồng thời, để ngăn chặn rủi ro bị truy cập trái phép từ các bộ phân phối CloudFront thuộc tài khoản AWS lạ, chính sách bắt buộc định danh nguồn (AWS:SourceArn) của yêu cầu phải khớp chính xác với mã ARN của bộ phân phối CloudFront được triển khai ở trên. Mã JSON cấu hình thực tế được áp dụng như sau:

```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::soc-dashboard-project2-alien/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::336457598104:distribution/E2MOMI2UBRLMBG"
        }
      }
    }
  ]
}
```

5.3 Triển khai Tầng Xử lý (Compute Tier)

Tầng xử lý đảm nhiệm vai trò điều phối logic toàn hệ thống, chia tách thành luồng thu thập dữ liệu tự động từ máy trạm và luồng cung cấp dịch vụ API cho giao diện quản trị viên.

Thành phần này được xây dựng hoàn toàn trên kiến trúc Serverless nhằm tối ưu hóa khả năng co giãn và triệt tiêu chi phí hạ tầng nhân rồi.

5.3.1 Cấu hình Hạ tầng Quản lý Định danh Amazon Cognito

Để bảo vệ các cổng API quản trị khỏi các truy cập trái phép, hệ thống triển khai dịch vụ Amazon Cognito User Pool đóng vai trò là nhà cung cấp danh tính tập trung (Identity Provider):

- **Cấu hình User Pool:** Khởi tạo một cơ sở dữ liệu người dùng khép kín dành riêng cho đội ngũ quản trị viên nội bộ (SOC Analysts). Chính sách mật khẩu được thiết lập nghiêm ngặt (tối thiểu 8 ký tự, bắt buộc chứa ký tự hoa, số và ký tự đặc biệt) kết hợp cơ chế xác thực đa yếu tố (MFA).
- **Cấu hình App Client và Hosted UI:** Thiết lập một ứng dụng máy khách (App Client) tích hợp giao diện đăng nhập mặc định (Hosted UI) do AWS cung cấp. Luồng xác thực áp dụng mô hình *OAuth 2.0 Authorization Code Grant*. Sau khi Admin xác thực thành công, Cognito sẽ cấp phát chuỗi mã thông báo tiêu chuẩn dạng JWT (bao gồm ID Token, Access Token và Refresh Token) để lưu trữ tại Local Storage phía trình duyệt Frontend.

5.3.2 Phát triển và Triển khai các Hàm AWS Lambda

Hệ thống sử dụng môi trường runtime **Python 3.14** hoạt động ở chế độ *On-Demand*. Nhằm giảm hiện tượng khởi động lạnh (Cold Start) đặc trưng của Serverless mà không làm phát sinh chi phí giữ ấm máy chủ, tính năng **AWS Lambda SnapStart** được kích hoạt trên các hàm xử lý chính.

- **Hàm LogIngestionController (Tập `lambda_function.py`):** Đảm nhiệm luồng tiếp nhận dữ liệu đầu vào. Hệ thống thực hiện xác thực chữ ký bảo mật của thiết bị đầu cuối, bóc tách chuỗi thông tin bằng biểu thức chính quy (Regex), và phân luồng ghi dữ liệu thô vào các bảng cơ sở dữ liệu DynamoDB tương ứng.
- **Hàm SOC_Query_Controller (Tập `RealtimeLogsFetcher.py` và `AdvancedSearchAPI.py`):** Phụ trách luồng truy xuất dữ liệu cho giao diện giám sát. Hàm tiếp nhận yêu cầu từ Dashboard, thực hiện các truy vấn thông qua chỉ mục phụ toàn cục (GSI) để trích xuất dữ liệu nhật ký theo thời gian thực (Realtime) và cung cấp các bộ lọc tra cứu lịch sử nâng cao.
- **Hàm SOC_Device_Manager (Tập `AlertStatusUpdater.py` và `DeviceCRUD_Manager.py`):** Quản lý trạng thái và danh mục hệ thống. Hàm xử lý các lệnh cập nhật trạng thái cảnh báo an ninh từ quản trị viên (chuyển sự cố sang trạng thái RESOLVED) và thực thi các thao tác quản trị thiết bị trạm (Thêm mới, Khóa, Xóa bỏ).

5.3.3 Thiết lập Phân quyền qua AWS IAM Roles

Để bảo đảm an toàn tuyệt đối cho hệ thống, mỗi hàm Lambda được gắn kèm một chính sách thực thi riêng biệt (*Execution IAM Role*).

1. Khởi tạo IAM Role và cấu hình Policy trên AWS Console

Quy trình khởi tạo vai trò thực thi (Execution Role) và đính kèm chính sách phân quyền cho các hàm Lambda được thực hiện tuần tự theo các bước sau:

1. Truy cập vào dịch vụ **IAM (Identity and Access Management)** → Chọn mục **Roles** tại thanh điều hướng bên trái → Nhấn nút **Create role**.
2. Tại mục **Trusted entity type**, chọn **AWS service**. Tại mục **Service or use case**, chọn **Lambda** trong danh sách thả xuống để cho phép hàm Lambda có quyền tương tác với các tài nguyên khác. Nhấn **Next**.
3. Tại bước **Add permissions**, tiến hành tìm kiếm và tích chọn chính sách mặc định **AWSLambdaBasicExecutionRole** để cấp quyền ghi dữ liệu hệ thống cơ bản lên CloudWatch Logs. Nhấn **Next**.
4. Tại bước **Name, review, and create**, đặt tên gợi nhớ cho vai trò thực thi tại mục **Role name**, sau đó nhấn **Create role** để hoàn tất khởi tạo.
5. Quay lại danh sách Roles, tìm kiếm và chọn đúng vào tên Role vừa tạo. Tại tab **Permissions**, nhấn vào nút **Add permissions** → Chọn **Create inline policy**.
6. Chuyển giao diện thiết lập sang chế độ **JSON**, tiến hành xóa bỏ nội dung cấu hình mặc định và dán đoạn mã chính sách phân quyền thực tế của hàm vào khung soạn thảo.
7. Nhấn **Next**, đặt tên cho chính sách (*Policy Name*) và nhấn **Save** để hoàn tất việc áp dụng đặc quyền tối thiểu cho vai trò.

2. Cấu hình IAM Role dành cho hàm LogIngestionController

Hàm này chỉ có nhiệm vụ đọc thông tin thiết bị để kiểm tra quyền và ghi log vào hệ thống. Quyền chỉnh sửa cấu hình thiết bị hay xóa dữ liệu bị cấm hoàn toàn để ngăn chặn kẻ tấn công chiếm quyền Lambda nhằm xóa dấu vết.

Tài nguyên mục tiêu	Hành động cho phép	Mục đích kỹ thuật
Bảng RegisteredDevices	dynamodb:GetItem	Đọc thông tin thiết bị để đối chiếu cặp khóa Machine-Token / Secret.
Bảng SystemLogs, Bảng FirewallLogs, Bảng SecurityAlerts	dynamodb:PutItem dynamodb:BatchWriteItem	Đẩy đồng thời các lô dữ liệu (25 records) xuống tầng lưu trữ sau khi trích xuất và phân tách cấu trúc.
Tài nguyên CloudWatch Logs	logs:CreateLogGroup logs:CreateLogStream logs:PutLogEvents	Cho phép hàm Lambda tự động tạo luồng log nội bộ để theo dõi trạng thái thực thi hệ thống.

Bảng 5.5: Chính sách phân quyền chi tiết cho hàm LogIngestionController

Mã cấu hình JSON thực tế được áp dụng cho Inline Policy của LogIngestionController:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:ap-southeast-1:*:table/RegisteredDevices"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:BatchWriteItem"
      ]
    }
  ]
}
```

```

    ],
    "Resource": [
        "arn:aws:dynamodb:ap-southeast-1:*:table/SystemLogs",
        "arn:aws:dynamodb:ap-southeast-1:*:table/FirewallLogs",
        "arn:aws:dynamodb:ap-southeast-1:*:table/SecurityAlerts"
    ]
}
]
}

```

3. IAM Role dành cho hàm SOC_Query_Controller

Đảm bảo hàm phục vụ Dashboard chỉ có đặc quyền đọc dữ liệu, ngăn chặn nguy cơ ghi đè hoặc thay đổi dữ liệu từ luồng tra cứu.

Tài nguyên mục tiêu	Hành động cho phép	Mục đích kỹ thuật
Toàn bộ các bảng Log và bảng Thiết bị	dynamodb:Query dynamodb:Scan	Thực hiện truy vấn danh sách log, tìm kiếm dữ liệu phục vụ hiển thị trên Dashboard.
Chỉ mục realtimeFetch của các bảng dữ liệu	dynamodb:Query	Truy xuất trực tiếp các sự kiện mới nhất dựa trên thuộc tính phân loại log_type.
Chỉ mục StatusIndex	dynamodb:Query	Lấy danh sách các cảnh báo bảo mật mới chưa được xử lý (NEW) từ bảng SecurityAlerts.

Bảng 5.6: Chính sách phân quyền chi tiết cho hàm SOC_Query_Controller

Mã cấu hình JSON thực tế được áp dụng cho Inline Policy của SOC_Query_Controller:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowQueryAndScanLogs",
      "Effect": "Allow",
      "Action": [
        "dynamodb:Query",
        "dynamodb:Scan"
      ]
    }
  ]
}

```

```

    ],
    "Resource": [
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/SystemLogs",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/SystemLogs/index/realtimeFetch",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/FirewallLogs",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/FirewallLogs/index/realtimeFetch",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/SecurityAlerts",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/SecurityAlerts/index/realtimeFetch",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/SecurityAlerts/index/alert_status-timestamp-index",
        "arn:aws:dynamodb:ap-southeast-1:336457598104:table/RegisteredDevices"
    ]
}
]
}

```

4. IAM Role dành cho hàm SOC_Device_Manager

Cấp phát các quyền chỉnh sửa cục bộ có giới hạn phục vụ các luồng tương tác quản trị trên Dashboard hệ thống.

Tài nguyên mục tiêu	Hành động cho phép	Mục đích kỹ thuật
Bảng SecurityAlerts	dynamodb:UpdateItem	Chuyển đổi trạng thái cột alert_status từ NEW sang RESOLVED khi Admin duyệt xử lý sự cố.
Bảng RegisteredDevices	dynamodb:PutItem dynamodb:UpdateItem dynamodb>DeleteItem dynamodb:GetItem dynamodb:Scan	Lấy thông tin chi tiết danh sách thiết bị, đăng ký máy trạm mới, khóa thiết bị hoặc xóa thiết bị khỏi hệ thống mạng.

Bảng 5.7: Chính sách phân quyền chi tiết cho hàm SOC_Device_Manager

Mã cấu hình JSON thực tế được áp dụng cho Inline Policy của SOC_Device_Manager:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowUpdateAlertStatus",
      "Effect": "Allow",
      "Action": [
        "dynamodb:UpdateItem"
      ],
      "Resource": "arn:aws:dynamodb:ap-southeast-1:336457598104:table/SecurityAlerts"
    },
    {
      "Sid": "AllowDeviceCRUD",
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb>DeleteItem",
        "dynamodb:GetItem",
        "dynamodb:Scan"
      ],
      "Resource": "arn:aws:dynamodb:ap-southeast-1:336457598104:table/RegisteredDevices"
    }
  ]
}

```

5.4 Triển khai Tầng Mạng biên (Edge Tier)

Tầng mạng biên đóng vai trò là tuyến phòng thủ đầu tiên (First Line of Defense) và là cổng giao tiếp duy nhất nối liền người dùng/máy trạm với các dịch vụ bên trong đám mây. Tầng này bao gồm hai dịch vụ lõi: Amazon API Gateway xử lý luồng dữ liệu động (Dynamic API Routing) và Amazon CloudFront xử lý luồng nội dung tĩnh (Static Content Delivery).

5.4.1 Cấu hình API Gateway và Định tuyến

Để tối ưu hóa hiệu năng và cô lập luồng dữ liệu, hệ thống triển khai hai cụm Amazon API Gateway độc lập: **SOC-Ingestion-API** (xử lý luồng log từ Agent) và **SOC-Admin-API** (xử lý luồng truy vấn từ Dashboard).

API Gateway	Endpoint (Method/Path)	Hàm xử lý / Bảo mật
SOC-Ingestion-API	POST /logs	LogIngestionController (IP Whitelist)
SOC-Admin-API	GET /dashboard/realtime, /dashboard/alerts, /dashboard/firewall, /dashboard/system	SOC_Query_Controller (Cognito Authorizer)
SOC-Admin-API	PUT /alerts/status, /devices/*	SOC_Device_Manager (Cognito Authorizer)

Bảng 5.8: Danh sách Endpoint và cơ chế bảo mật định tuyến trên API Gateway

5.4.2 Thiết lập Bộ lọc IP (IP Whitelisting) qua Resource Policy

Đối với luồng Ingestion (nhận log từ máy trạm qua đường dẫn /ingest), để ngăn chặn các nỗ lực tấn công từ chối dịch vụ (DDoS) hoặc quét cổng (Port Scanning) rác từ Internet, chính sách tài nguyên (Resource Policy) được áp dụng trực tiếp lên API Gateway.

Cơ chế này sử dụng điều kiện **NotIpAddress** để chặn mọi luồng truy cập có địa chỉ IP không nằm trong danh sách an toàn đã khai báo. Bất kỳ máy khách lạ nào cố tình gọi API sẽ lập tức nhận mã lỗi **HTTP 403 Forbidden** từ Edge Node của AWS mà không tiêu tốn tài nguyên xử lý của hàm Lambda phía sau.

Đoạn mã JSON cấu hình chính sách chặn IP thực tế trên hệ thống:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:ap-southeast-1:336457598104:y288ko9o4j/*/*/*"
    },
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:ap-southeast-1:336457598104:y288ko9o4j/*/*/*",
      "Condition": {
        "NotIpAddress": {
```

```

        "aws:SourceIp": [
            "113.23.48.184/32" ,
            "203.210.148.0/24"
        ]
    }
}
]
}

```

5.4.3 Tích hợp Trình xác thực Amazon Cognito Authorizer

Đối với các endpoint phục vụ Dashboard quản trị (/dashboard, /alerts, /devices), hệ thống không sử dụng IP Whitelist (do quản trị viên có thể truy cập từ nhiều mạng khác nhau) mà thiết lập **Cognito Authorizer**.

- Tại API Gateway, các endpoint quản trị được cấu hình sử dụng *Cognito Authorizer* thông qua việc tạo Authorizer (mục **Authorizers**) và liên kết với User Pool đã tạo.
- Quản trị viên sau khi đăng nhập thành công sẽ nhận được một chuỗi JSON Web Token (JWT).
- Mỗi khi giao diện Dashboard gọi API, chuỗi Access Token này sẽ được tự động kẹp vào header *Authorization* của request.
- API Gateway sẽ chặn luồng request, tự động giải mã và kiểm tra tính hợp lệ của token với dịch vụ Cognito. Nếu token sai, hết hạn hoặc bị giả mạo, request bị từ chối ngay tại cổng viên. Nếu hợp lệ, thông tin định danh sẽ được chuyển tiếp vào hàm Lambda.

5.4.4 Thiết lập Amazon CloudFront và Bảo mật SSL/HTTPS

Mặc dù mã nguồn tĩnh của giao diện quản trị (ReactJS) được lưu trữ trên S3, nhưng S3 tĩnh không hỗ trợ giao thức bảo mật HTTPS. Việc tải nội dung qua HTTP thông thường sẽ gây ra lỗi *Mixed Content* (trình duyệt chặn website HTTP gọi dữ liệu từ API Gateway HTTPS), đồng thời làm lộ token JWT trên đường truyền.

Để giải quyết vấn đề này, hệ thống triển khai **Amazon CloudFront (CDN)** làm mạng phân phối nội dung với các cấu hình sau:

- **Cấp phát chứng chỉ SSL/TLS:** Gắn chứng chỉ điện tử miễn phí từ AWS Certificate Manager (ACM) vào CloudFront, tạo ra một đường hầm mã hóa dữ liệu HTTPS từ đầu cuối trình duyệt đến máy chủ biên. Trình duyệt bắt buộc (Force HTTPS) người dùng điều hướng sang kết nối an toàn.

- **Định tuyến tới nguồn S3 (Origin):** Cấu hình CloudFront đọc dữ liệu gốc từ S3 Bucket thông qua tính năng xác thực truy cập nội bộ (Origin Access Control - OAC).
- **Phản hồi tiêu đề bảo mật (Security Headers):** CloudFront được cấu hình để thêm tự động các tiêu đề bảo mật (như HSTS, CORS) vào mọi gói tin trả về, đảm bảo ứng dụng ReactJS có thể giao tiếp mượt mà với API Gateway và Cognito.

CHƯƠNG 6.

TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN (CONCLUSION & FUTURE WORK)

6.1 Kết luận

Đồ án đã hoàn thành việc phân tích, thiết kế và triển khai thực nghiệm một hệ thống Thu thập, Truy vết và Phân tích Sự kiện Mạng (Log) Tập trung dựa trên kiến trúc Serverless toàn diện trên nền tảng Amazon Web Services. Đối chiếu với bốn mục tiêu cốt lõi đặt ra ban đầu, hệ thống đã đạt được các kết quả cụ thể như sau:

Thứ nhất, về kênh truyền tải an toàn tại tầng mạng biên: Hệ thống đã thiết lập thành công cơ chế lọc giao thông hai lớp tại API Gateway. Lớp ngoài cùng sử dụng Resource Policy kiểm soát IP Whitelist, chặn đứng toàn bộ kết nối từ các nguồn không xác định ngay trước khi traffic chạm tới tầng xử lý. Lớp thứ hai xác thực định danh thiết bị thông qua cặp khóa Machine-Token và Machine-Secret đính kèm trong HTTP Header, loại bỏ nguy cơ giả mạo dữ liệu nhật ký (Log Spoofing) từ các máy trạm lạ. Sự kết hợp hai lớp kiểm soát này đảm bảo chỉ các thiết bị đã được cấp phép và đang hoạt động (ACTIVE) mới có thể đẩy dữ liệu lên Cloud.

Thứ hai, về kiến trúc xử lý phân tán (Backend Logic Tier): Ba cụm điều khiển Lambda độc lập (LogIngestionController, QueryController, ManagerController) đã được xây dựng với ranh giới trách nhiệm rõ ràng, đảm bảo cô lập lỗi. Đáng chú ý, LogIngestionController hoạt động theo nguyên tắc ghi một chiều: IAM Execution Role chỉ cấp quyền BatchWriteItem mà không cấp UpdateItem hay DeleteItem, đảm bảo toàn vẹn bằng chứng số ngay cả trong trường hợp hàm Lambda bị kẻ tấn công kiểm soát. Cơ chế xác thực JWT tập trung tại Cognito Authorizer trên API Gateway giúp bảo vệ đồng nhất toàn bộ nhóm API quản trị mà không cần lặp lại logic xác thực trong từng hàm Lambda.

Thứ ba, về mô hình cơ sở dữ liệu NoSQL: Lược đồ bốn bảng (RegisteredDevices, SecurityAlerts, FirewallLogs, SystemLogs) với cấu hình On-Demand Capacity đã đáp ứng yêu cầu ghi cường độ cao mà không cần dự trữ tài nguyên tĩnh. Các GSI được thiết kế có chủ đích, phục vụ hai luồng truy vấn trái chiều nhau: truy vấn nhanh 50 bản ghi mới nhất cho

màn hình Realtime và truy vấn có điều kiện theo khoảng thời gian cho luồng điều tra lịch sử, tất cả đều tránh được thao tác Scan toàn bảng tốn kém.

Thứ tư, về giao diện giám sát và hạ tầng phân phối: Dashboard ReactJS với cơ chế Auto-refresh mỗi 1,5 giây đã cung cấp luồng dữ liệu cảnh báo gần như tức thời cho quản trị viên. Giao diện quản lý thiết bị tích hợp cơ chế bảo vệ chống thao tác nhầm (xác nhận hai bước với bộ đếm ngược 10 giây khi xóa). Toàn bộ mã nguồn Frontend được phân phối qua CloudFront với OAC, ép buộc HTTPS và che khuất hoàn toàn URL gốc của S3 bucket.

Nhìn lại toàn bộ quá trình, điểm cốt lõi mà kiến trúc Serverless mang lại không chỉ là giảm chi phí vận hành mà còn là **chuyển dịch mô hình bảo mật**: thay vì bảo vệ một perimeter mạng với địa chỉ IP công khai dễ bị rà quét, hệ thống sử dụng **định danh (Identity) làm ranh giới**. Mỗi lớp dịch vụ chỉ tin tưởng lớp ngay phía trên nó thông qua cơ chế xác thực riêng (OAC, IAM Role, JWT, Token/Secret), tạo nên một chuỗi tin cậy (Chain of Trust) khép kín mà không cần VPC hay firewall truyền thống.

6.2 Hướng phát triển

Trong khuôn khổ đề án, một số tính năng và cải tiến kỹ thuật chưa được triển khai do giới hạn về thời gian. Các hướng mở rộng được đề xuất dưới đây nhằm nâng tầm hệ thống từ bản thực nghiệm (prototype) lên môi trường vận hành chuyên nghiệp (production-grade):

1. **Tự động hóa triển khai hạ tầng (Infrastructure as Code):** Toàn bộ quá trình khởi tạo hạ tầng hiện tại được thực hiện thủ công qua AWS Management Console, tiềm ẩn rủi ro sai sót cấu hình và không thể tái lập nhất quán. Hướng phát triển tiếp theo là chuyển đổi hoàn toàn sang **Terraform** để mã hóa toàn bộ hạ tầng dưới dạng code (.tf), bao gồm: cấu hình IAM Role, Resource Policy, Cognito User Pool, DynamoDB table schema, CloudFront distribution và API Gateway routes. Cách tiếp cận này cho phép kiểm soát phiên bản cấu hình qua Git, tái triển khai toàn bộ hệ thống trong vài phút và loại bỏ rủi ro “configuration drift” trong môi trường vận hành dài hạn.
2. **Hệ thống cảnh báo chủ động (Proactive Alerting):** Giao diện Dashboard hiện tại yêu cầu quản trị viên phải chủ động theo dõi màn hình để phát hiện cảnh báo mới. Cần bổ sung luồng thông báo chủ động bằng cách tích hợp **Amazon SNS** (Simple Notification Service): khi Lambda ghi một bản ghi mới vào bảng SecurityAlerts với `severity_level = CRITICAL`, hệ thống tự động kích hoạt SNS topic để gửi thông báo tức thì qua email hoặc SMS đến quản trị viên trực ca, rút ngắn thời gian phản ứng sự cố (MTTR) mà không đòi hỏi admin phải liên tục theo dõi màn hình.
3. **Phát hiện bất thường bằng AI/ML:** Luồng phát hiện tấn công hiện tại dựa hoàn toàn vào tập luật (rule-based) tĩnh, vốn không nhận diện được các mối đe dọa mới (zero-day)

hoặc các chiến thuật tấn công tinh vi. Hướng phát triển đề xuất tích hợp hai dịch vụ AWS:

- **Amazon GuardDuty:** Phân tích hành vi mạng tổng thể (VPC Flow Logs, DNS logs) để tự động phát hiện các mẫu bất thường như quét cổng có chủ đích, liên lạc với các C&C server đã biết, hoặc hành vi leo thang đặc quyền nội bộ – những điều mà rule-based IDS khó phát hiện.
- **Amazon Macie:** Áp dụng Machine Learning để quét và phân loại nội dung dữ liệu log, tự động gắn cờ khi phát hiện các mẫu thông tin nhạy cảm (số thẻ tín dụng, khóa API) xuất hiện trong nhật ký, hỗ trợ yêu cầu tuân thủ bảo mật dữ liệu (DLP – Data Loss Prevention).

4. **Mở rộng nguồn dữ liệu và tương quan đa nguồn (Multi-source Correlation):** Hệ thống hiện chỉ tiếp nhận log từ các máy trạm Linux chạy Agent Python. Để xây dựng bức tranh tổng thể về tình trạng an ninh của toàn bộ hạ tầng, cần mở rộng theo hai hướng:

- **Đa nền tảng Agent:** Phát triển Agent cho hệ điều hành Windows (thu thập Windows Event Log, PowerShell audit log) và hỗ trợ giao thức Syslog/SNMP để tích hợp log từ các thiết bị mạng (Router, Switch, Firewall vật lý).
- **Tương quan đa nguồn:** Xây dựng một lớp phân tích (CorrelationEngine) tại Lambda có khả năng liên kết các sự kiện rời rạc từ nhiều bảng log với nhau theo cùng khoảng thời gian và địa chỉ IP nguồn. Ví dụ: phát hiện chuỗi “SSH Brute-force (SystemLogs) → Đăng nhập thành công (SystemLogs) → Quét cổng nội bộ (FirewallLogs)” như một sự cố leo thang đặc quyền có chủ đích thay vì ba sự kiện đơn lẻ không liên quan.